

Sparse Koopman Supports for Multibasin Forecasting and Transductive Regime Alignment

Anonymous Authors

Abstract

Koopman autoencoders (KAEs) seek a higher-dimensional latent representation in which nonlinear dynamics evolve linearly. However, systems with multiple basins need not admit an exact, state-reconstructing finite-dimensional Koopman embedding with one continuous global coordinate map, so a single-transition KAE may have to approximate across dynamically different regions. We posit that sparse encoders can expose latent supports as inspectable, model-produced regime variables without basin labels or other regime annotations during training. On 15 controlled multibasin systems and a 10-system chaotic-flow stress test, sparse-latent KAEs forecast more accurately than the reported dense-latent KAE controls, although decoder normalization differs across encoder families. In a post-hoc transductive diagnostic on the 14 systems whose evaluation assignment realizes at least two labels, LISTA support families carry substantially more information about native or nearest-center-proxy labels than the dense-latent control’s thresholded support families. The remaining catalog proxy collapses to one observed label; it is excluded from primary aggregation but retained in raw and per-system evidence and in an all-15-system sensitivity. Finally, on the controlled benchmark, support families learned after the first training stage route a second-stage bank of local affine latent models and yield descriptively lower forecasting error than a same-architecture, same-budget global- K LISTA baseline; this comparison is optimistic because the staged checkpoint selector overlaps the reported evaluation starts and differs from the baseline selector. Together these results provide bounded evidence that sparse supports can be useful label-free regime variables for Koopman prediction; they do not establish a global exact linearization or out-of-sample basin discovery.

1 Introduction

Learning representations for nonlinear dynamical systems remains a central problem in machine learning, scientific computing, and control. A particularly useful goal is to embed nonlinear dynamics into coordinates in which the evolution is linear, or at least locally linear, because linear models can be combined with mature tools for prediction, estimation, and control, including model predictive control and linear quadratic regulation (Mayne et al., 2000; Grüne and Pannek, 2017; Korda and Mezić, 2018; Kalman, 1960; Mamakoukas et al., 2019). This goal is also well motivated theoretically. Classical dynamical-systems results provide local topological linearizations near hyperbolic fixed points (Grobman, 1959; Hartman, 1960), while Koopman eigenfunction constructions can yield linearizing coordinates on basins of attraction for stable equilibria and periodic orbits (Lan and Mezić, 2013; Kvalheim and Revzen, 2021). Thus, nonlinear systems may admit linear dynamics with a specific change of coordinates.

The difficulty is that these results are primarily existential. They establish when useful coordinates can exist, but they do not by themselves provide a finite-data procedure for constructing such coordinates. Koopman operator theory formalizes the linearization idea by lifting nonlinear state evolution to a linear operator acting on observables (Koopman, 1931; Koopman and Neumann,

1932; Mezić, 2005; Brunton et al., 2022). This viewpoint has inspired data-driven approximations of Koopman dynamics, such as DMD and eDMD (Schmid, 2010; Williams et al., 2015, 2016), and more recently to Koopman autoencoders, which learn an encoder from state space to a latent representation, a linear latent transition matrix, and a decoder back to state space (Takeishi et al., 2017; Lusch et al., 2018; Otto and Rowley, 2019; Azencot et al., 2020; Shi and Meng, 2022; Fathi et al., 2024). These models provide an appealing route from existence results to practical learning: rather than hand-designing observables, they attempt to learn the embedding directly from trajectory data.

A central challenge for Koopman approximations is that many practically interesting nonlinear systems typically contain multiple equilibria or basins of attraction; for example, a damped mechanical system can settle into different wells, a biological or chemical model can approach different stable equilibria, and a controlled system can move between regions with distinct local dynamics. Theory places strong restrictions on exact finite-dimensional global linearizations with continuous, state-reconstructing encoder-decoder pairs in multi-attractor settings (Lan and Mezić, 2013; Brunton et al., 2016; Pan and Duraisamy, 2024; Liu et al., 2025), and empirical work documents practical difficulty for a single global model (Fathi et al., 2024). Local or basin-restricted linear descriptions may still be useful, but their simultaneous representation by one finite matrix is not guaranteed. This motivates testing whether a learned representation can expose local predictive structure without assuming that every basin shares one exact global linearization.

This paper proposes inducing sparse latent structure as a practical and interpretable method for Koopman learning of multi-attractor systems. By inducing the state to activate only a small subset of coordinates in lifted latent space, we can incentivize different regions of state to occupy different active coordinates, since nearby states should have similar latent embeddings, and thus enable local linearizations. In this sparse representation, nonzero coefficient values in a latent vector can provide continuous coordinates for prediction within a selected region, while the active indices provide a discrete support object that can be inspected across states and used to select local linearizations.

Sparse coding and LISTA-style encoders provide a plausible inductive bias for this purpose. They induce piecewise-linear, union-of-subspaces structure (Olshausen and Field, 1996; Chen et al., 1998; Donoho and Elad, 2003; Gregor and LeCun, 2010), which may help when useful predictive descriptions differ across state-space regions. We hypothesize that sparse-latent Koopman autoencoders will assign local dynamical regimes to distinct latent support families, yielding a learned regime variable without basin supervision. Periodically decoding and re-encoding the model’s prediction can then refresh the inferred latent support as a rollout moves across state space (Fathi et al., 2024).

Contributions. This paper makes three empirical contributions. First, across controlled multi-basin systems and an external chaotic-flow stress test, sparse-latent KAEs improve rollout accuracy over the reported dense-latent KAE controls. Second, a post-hoc transductive diagnostic on 14 informative controlled systems shows that LISTA absolute-threshold support families align with evaluation-only native or nearest-center-proxy labels on high center-margin states, without using those labels to train the model or as features in the mask merge. Third, on the controlled benchmark, support families formed after the first stage of LISTA training route a second-stage bank of local affine latent predictors and produce descriptively lower error than a same-architecture, same-budget global- K LISTA model under an asymmetric, partly evaluation-overlapping checkpoint-selection protocol. These are forecasting, alignment, and descriptive routing claims, respectively. Unstructured LISTA, Sparse MLP, Dense MLP, support-family formation, and staged routing use no basin information during training or rollout; the structured-transition diagnostic rows are the exception, using a known count only to size their blocks.

2 Problem formulation

The introduction motivates a multibasin version of Koopman learning. Given sampled trajectories of a nonlinear system, the aim is to learn a state-reconstructing latent representation whose evolution is linear, without being told which basin or local dynamical regime generated each observation. We formulate the problem at the cadence of the stored benchmark observations. For each system, let $\mathcal{X} \subseteq \mathbb{R}^{d_x}$ be the observed state space and let

$$x_{k+1} = F_{\Delta t}(x_k), \quad x_k \in \mathcal{X}, \quad (1)$$

where $x_k \in \mathbb{R}^{d_x}$ is the state at the k th stored sample, $F_{\Delta t}$ denotes the transformation to the next discrete observation step based on underlying dynamics, and Δt is the system-specific observation interval. Forecasting is therefore defined by repeatedly applying the map $F_{\Delta t}$; a horizon of h means h applications of $F_{\Delta t}$; we denote k compositions as $F_{\Delta t}^{(k)}$. Because Δt may differ across systems, equal values of h may yield different physical time.

Basins of attraction. Many systems of interest are multistable: different initial conditions can approach different long-run behaviors. A fixed point is a state x^* satisfying $F_{\Delta t}(x^*) = x^*$. More generally, a trajectory may approach an attractor A , such as a stable equilibrium, limit cycle, countable finite set, or chaotic invariant set. The basin of attraction of A is the set of initial conditions whose trajectories converge to A :

$$\mathcal{B}(A) = \left\{ x_0 \in \mathbb{R}^{d_x} : \inf_{a \in A} \left\| F_{\Delta t}^{(k)}(x_0) - a \right\|_2 \rightarrow 0 \text{ as } k \rightarrow \infty \right\}. \quad (2)$$

When multiple attractors coexist, their basins partition the state space up to basin boundaries. This defines the setup for this paper. The task is to learn a representation that can predict while preserving inspectable information about where the state lies in the multibasin geometry.

Koopman theory. The stored-step map $F_{\Delta t}$ induces a Koopman operator $\mathcal{K}$ on scalar observables $g : \mathcal{X} \rightarrow \mathbb{R}$ by composition, $(\mathcal{K}g)(x) = g(F_{\Delta t}(x))$. Although $F_{\Delta t}$ may be nonlinear, $\mathcal{K}$ is linear on the space of observables. A finite-dimensional Koopman approximation searches for a vector observable $\Phi : \mathcal{X} \rightarrow \mathbb{R}^{d_z}$ and a matrix K such that $\Phi(F_{\Delta t}(x)) \approx K\Phi(x)$, while retaining enough information to reconstruct the state. Additional background is provided in [section I](#).

Koopman autoencoders. We study this setting with Koopman autoencoders (KAEs). The model learns an encoder $E_\theta : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_z}$, a decoder $D_\phi : \mathbb{R}^{d_z} \rightarrow \mathbb{R}^{d_x}$, and a latent linear transition matrix $K \in \mathbb{R}^{d_z \times d_z}$. For observation x_k at arbitrary timestep k , the corresponding open-loop h -step forecast is

$$\hat{x}_{k+h} = D_\phi \left(K^h E_\theta(x_k) \right). \quad (3)$$

The same matrix K is applied at every step of the rollout, so any state-dependent structure required for prediction must be encoded in the learned representation rather than in a supplied regime label.

Training windows. Training uses windows of adjacent stored observations. For a batch of B windows of rollout length L , $\{x_{b,k}, x_{b,k+1}, \dots, x_{b,k+L}\}_{b=1}^B$, where b indexes windows and ℓ indexes offsets within a window, the model encodes each observed state, rolls out from the initial encoded

state, and decodes both reconstructed observations and latent forecasts. The encoded/reconstructed quantities are defined for $\ell = 0, \dots, L$, while rollout/forecast quantities are defined for $\ell = 1, \dots, L$:

$$z_{b,k+\ell}^{\text{enc}} = E_{\theta}(x_{b,k+\ell}), \quad z_{b,k+\ell}^{\text{roll}} = K^{\ell} z_{b,k}^{\text{enc}}, \quad x_{b,k+\ell}^{\text{rec}} = D_{\phi}(z_{b,k+\ell}^{\text{enc}}), \quad \hat{x}_{b,k+\ell} = D_{\phi}(z_{b,k+\ell}^{\text{roll}}). \quad (4)$$

The training losses described later are built from these reconstructed and rolled-out quantities. Each application of K is interpreted as one stored benchmark step, matching the observation map in [equation \(1\)](#).

Label-free training and rollout. In the intended training and deployment setting, the sparse-support mechanism is not given basin labels, the number of basins, or trajectory-to-basin assignments. Native labels and centers, when available, are used only for post-hoc evaluation; for the 13 catalog systems without them, the known benchmark count is used after training to estimate centers and construct nearest-center proxy labels. The block-diagonal and soft-block rows are a separate architecture-size exception: on the controlled benchmarks, their number of transition blocks is set equal to the benchmark basin count; on Dysts, it is set from a hand-specified lobe, scroll, or equilibrium count. No label or count enters support-family formation or routing, as detailed in [section A](#).

3 Method

3.1 Sparse supports as local regime variables

A finite-dimensional Koopman representation for the multi-basin systems in [section 2](#) should encode two complementary quantities: (i) continuous coordinates that linearize the dynamics within a basin, and (ii) indicate which local linearization is active. We use sparsity to couple these two roles. If the state x is represented by a sparse latent code z , then the support of z can act as a regime variable, while the nonzero coefficient values provide coordinates for the corresponding local linear representation.

Given an overcomplete dictionary D , sparse coding estimates z by solving the Lasso objective ([Beck and Teboulle, 2009](#))

$$z^{\star}(x) = \arg \min_{z \in \mathbb{R}^{dz}} \frac{1}{2} \|x - Dz\|_2^2 + \lambda_{\text{sc}} \|z\|_1, \quad \lambda_{\text{sc}} > 0. \quad (5)$$

The L_1 term is a convex surrogate that promotes sparse codes; it does not directly minimize the number of nonzero coefficients and need not recover the sparsest feasible code.

The classical solver to this problem (ISTA ([Beck and Teboulle, 2009](#))) can be unrolled into a depth- Q network of ISTA iterations, yielding learned ISTA (LISTA) ([Gregor and LeCun, 2010](#)); in our setting, LISTA is an encoder yielding the sparse code $z = E_{\theta}(x)$, where the *support* is the set of active z coordinates. With a linear decoder, the support determines which decoder columns participate in reconstructing x , and the associated nonzero coefficients specify coordinates within that selected reconstruction subspace. When D and z are learned jointly, as in sparse coding ([Olshausen and Field, 1996](#)), this induces a data-adaptive partition of the input state space where each region can be locally reconstructed with a code having the same support.

We therefore hypothesize that combining Koopman prediction with sparse coding may expose useful multibasin structure without explicit basin labels in training. The Koopman objective encourages continuous coefficients to support linear rollout, while the sparsity objective promotes selective activation; neither objective by itself guarantees that different regimes use different supports.

If that separation emerges, a support pattern can serve as a discrete route key while its nonzero coefficients carry continuous local predictive coordinates. When native basin labels are available in benchmarks, we use them only after training to assess support–basin alignment or to construct diagnostic interventions; otherwise the alignment evaluator constructs the nearest-center proxy labels described below.

3.2 Koopman autoencoder and sparse encoder

The predictor follows the KAE formulation in [section 2](#): an encoder, linear decoder, and latent transition, all of which are optimized jointly, produce the rollout in [equation \(3\)](#). The reported LISTA rows decode through a dictionary whose atoms are normalized. The reported sparse and dense MLP rows use ordinary learned linear decoders without that atom-normalization constraint. Decoder normalization is therefore not matched across encoder families and is a limitation of cross-family attribution. We experiment with K that is dense, block diagonal, or softly block-regularized.

Sparse encoding. Our main sparse encoder is LISTA ([Gregor and LeCun, 2010](#)). It first computes a dense pre-code $c_t = f_\theta(x_t)$, then applies learned shrinkage refinements for $q = 0, \dots, Q - 1$:

$$u_t^{(0)} = \text{shrink}(c_t, \tau), \quad u_t^{(q+1)} = \text{shrink}(S u_t^{(q)} + c_t, \tau), \quad z_t = u_t^{(Q)}. \quad (6)$$

Here, $\text{shrink}(a, \tau) = \text{sign}(a) \max(|a| - \tau, 0)$ is applied element-wise, S is learned, and τ controls the threshold scale for shrinkage. The thresholding step creates exact zeros, so the encoder explicitly selects active latent coordinates while learning the selection rule from the prediction objective.

Transition families. We vary the structure of K separately from the encoder. The dense transition leaves K unrestricted. The block-diagonal transition partitions latent coordinates into M fixed groups and constrains

$$K_{\text{block}} = \text{blkdiag}(K_1, \dots, K_M). \quad (7)$$

We also use a soft-block LISTA ablation that keeps K dense but penalizes cross-group entries,

$$\mathcal{L}_{\text{offblock}} = \sum_{i,j: g(i) \neq g(j)} |K_{ij}|, \quad (8)$$

where $g(i)$ is the fixed architectural group containing coordinate i .

3.3 Training objective

Training uses adjacent observation windows. For a batch of B windows with prediction horizon L , the model produces reconstructed future states $x_{b,k+\ell}^{\text{rec}}$, latent rollouts $z_{b,k+\ell}^{\text{roll}}$, encoded future latents $z_{b,k+\ell}^{\text{enc}}$, and decoded rollout predictions $\hat{x}_{b,k+\ell}$ as defined in [equation \(4\)](#). The rollout is seeded by the initial state, and the following sums are over offsets $\ell = 1, \dots, L$:

$$\bar{\mathcal{L}}_{\text{pred}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \|\hat{x}_{b,k+\ell} - x_{b,k+\ell}\|_2, \quad \bar{\mathcal{L}}_{\text{rec}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \|x_{b,k+\ell}^{\text{rec}} - x_{b,k+\ell}\|_2, \quad (9)$$

$$\bar{\mathcal{L}}_{\text{lin}} = \frac{1}{BL} \sum_{b,\ell} \|z_{b,k+\ell}^{\text{roll}} - z_{b,k+\ell}^{\text{enc}}\|_2, \quad \bar{\mathcal{L}}_{\text{sp}} = \frac{1}{BL} \sum_{b,\ell} \|z_{b,k+\ell}^{\text{roll}}\|_1. \quad (10)$$

The observation-space terms are normalized by $\sqrt{d_x}$ so that their scale is comparable across systems of different state dimension. The latent terms are left in their native scale because the latent dimension is fixed within each comparison.

The total objective is

$$\mathcal{L} = \lambda_{\text{pred}} \bar{\mathcal{L}}_{\text{pred}} + \lambda_{\text{rec}} \bar{\mathcal{L}}_{\text{rec}} + \lambda_{\text{lin}} \bar{\mathcal{L}}_{\text{lin}} + \lambda_{\text{sp}} \bar{\mathcal{L}}_{\text{sp}} + \mathcal{L}_{\text{struct}}. \quad (11)$$

Here $\mathcal{L}_{\text{struct}} = 0$ for the dense and block-diagonal transition variants. For the soft-block transition, $\mathcal{L}_{\text{struct}} = \lambda_{\text{off}} \mathcal{L}_{\text{offblock}}$, which probes whether a soft transition grouping is sufficient to induce useful support structure. The prediction and latent-linearity terms make the representation useful for Koopman rollout, the reconstruction term keeps the code tied to the observed state, and the sparsity term encourages selective activation.

3.4 Support objects

Let $z = E_\theta(x) \in \mathbb{R}^{d_z}$. A *support rule* converts z into a Boolean vector $m(x) \in \{0, 1\}^{d_z}$, where $m_i(x) = \mathbf{1}\{|z_i| > 0\}$. Equivalently, the exact support is $S(x) = \{i : m_i(x) = 1\}$. Exact supports defined by an arbitrary threshold can be unstable: a small perturbation in x -space can change the support, so one basin may be covered by several nearby or partially overlapping active-coordinate vectors. We therefore distinguish exact supports from *support families*, which group overlapping Boolean masks to reduce fragmentation. This construction does not by itself guarantee robustness or basin-scale meaning.

Both support-family experiments begin with $m_{\text{abs},i}(x) = \mathbf{1}\{|z_i| > 10^{-3}\}$ and $S_{\text{abs}}(x) = \{i : m_{\text{abs},i}(x) = 1\}$, but they use distinct fixed merging protocols. For the alignment diagnostic, denoted $F_{\text{abs}}^{\text{align}}$, we count distinct masks over all states in the evaluation trajectory collection and visit them from most to least frequent. Each family f stores one representative mask r_f ; a new mask m joins the family with the largest Jaccard score

$$J(m, r_f) = \frac{\sum_i \mathbf{1}\{m_i = 1 \text{ and } r_{f,i} = 1\}}{\sum_i \mathbf{1}\{m_i = 1 \text{ or } r_{f,i} = 1\}},$$

with $J(0, 0) = 1$, if that score is at least $\tau = 0.50$; otherwise it starts a new family. Evaluation-only labels and their native or endpoint-estimated centers then select a per-label high center-margin subset only for scoring $H(B \mid F_{\text{abs}}^{\text{align}})$; they do not select the masks used to form families or equalize label sample counts. The diagnostic is transductive because the partition is fit on the same evaluation trajectory collection containing the scored states, rather than on a disjoint fitting collection. The staged forecasting experiment instead fits $F_{\text{abs}}^{\text{route}}$ on training-distribution trajectories with $\tau = 0.40$. The two family partitions must not be treated as interchangeable. Pseudocode and implementation details are provided in [section D](#).

3.5 Periodic re-encoding

If a support indicates the currently relevant local dynamics, a long rollout should be able to update that support. We therefore evaluate periodic re-encoding, following [Fathi et al. \(2024\)](#). Starting from $z_k = E_\theta(x_k)$ and a re-encoding period m , the model advances for m Koopman steps, decodes the predicted state, and encodes that prediction again:

$$\tilde{z}_{k+m} = K^m z_k, \quad \tilde{x}_{k+m} = D_\phi(\tilde{z}_{k+m}), \quad z_{k+m} = E_\theta(\tilde{x}_{k+m}). \quad (12)$$

The same procedure is repeated over the rollout horizon. Each rollout uses only the model’s predicted state; it does not use the true future state, a basin label, or an oracle for basin transitions. However,

the reported “best-periodic” forecasting value is selected separately for each system, model, seed, and horizon by minimizing MSE on the same reported evaluation trajectories over a fixed cadence grid. This is oracle-over-grid test selection, not a validation-frozen deployment rule, and it makes the reported values optimistic.

4 Experiments

The experiments evaluate three questions: whether sparse-latent KAEs reduce forecasting error, whether their post-hoc support families align transductively with evaluation-only native or proxy basin labels, and whether a separately fitted training-distribution support-family route has lower descriptive controlled-forecasting error. A one-checkpoint coordinate intervention is retained as a scoped descriptive diagnostic.

Benchmark systems. The main benchmark contains 15 heterogeneous, author-defined analytic or piecewise-analytic two-dimensional vector fields with multiple attractors. It includes potential-based, locally gated, oscillatory, and bifurcation-derived constructions; exact equations and retained metadata are in Appendix B. The two gated environments expose native labels and centers for evaluation. The other 13 catalog environments expose neither, so the evaluator advances trajectory endpoints, estimates the manifest-specified number of centers by deterministic k -means, and assigns every evaluation state to its nearest estimated center. Native and proxy labels and centers never train any row or form support families. The known count is used only to size structured-transition diagnostic blocks and, after training, to estimate catalog-system centers; it is not a runtime routing feature. We also use a subset of 10 chaotic systems from the Dysts repository (Gilpin, 2021, 2023) with the step size dt multiplied by a factor of 30 as an additional forecasting test. Vector-field expressions and more details are in Appendix B, C.

Training models. We compare the six KAE models shown in the upper block of table 1 by varying the encoder and latent transition structure. We test three LISTA sparse encoders (where LISTA-BD denotes block diagonal latent transitions and LISTA-SB denotes soft-block regularization), two sparse-latent MLP controls applying the sparsity penalty $\tilde{\mathcal{L}}_{\text{sp}}$ (10) with dense (Fathi et al., 2024) or block-diagonal (BD) transitions, and a dense-latent MLP control baseline with no sparsity incentive. Each model is trained for 15 random seeds on each retained system. On Dysts, LISTA-SB is an architecture-specific ablation: at the same 100,000-step budget it uses two LISTA loops and a sign-split output, whereas LISTA and LISTA-BD use one loop and a ReLU output. Its run packet contained 12 systems; the table aggregates the fixed retained 10-system subset. Exact details are in Appendix A.

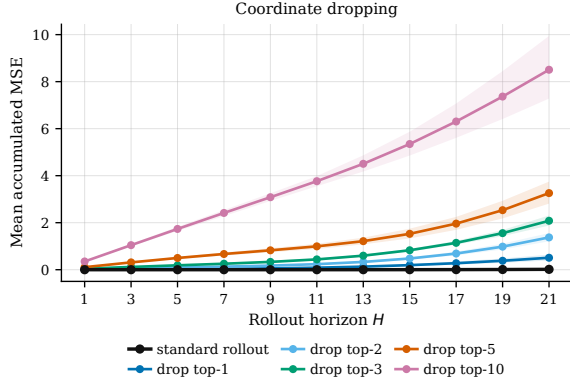
Statistical details. Point estimates use `scipy.stats.trim_mean` with proportion 0.25 over finite seeds within each system, then arithmetic means across systems. This implementation sorts n values and removes $\lfloor n/4 \rfloor$ from each tail; a complete 15-seed cell therefore averages its central nine values. We call this the interquartile mean (IQM) (Agarwal et al., 2021). The table annotations use paired tests and Holm corrections with the inferential units specified in Appendix F.

4.1 Forecasting experiments

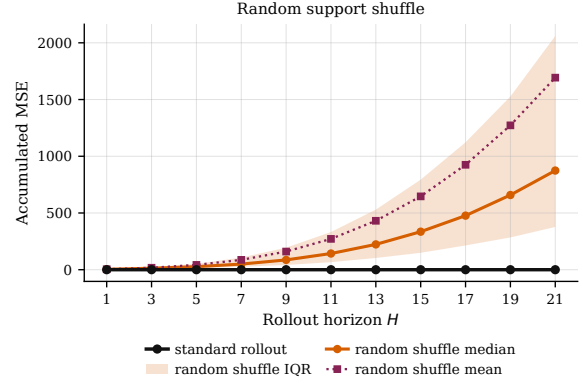
Sparse-latent KAEs reduce forecasting error. We evaluate trained sparse and dense-latent KAEs on the two benchmarks spanning 25 systems. Forecasting is evaluated on held-out trajectories

Table 1: Forecasting performance on the controlled multibasin and Dysts benchmarks; lower is better. Each KAE cell is an arithmetic mean across systems after a 25%-per-tail trimmed mean over seeds within each system; complete 15-seed cells average the central nine values. Superscripts mark Holm-corrected tests against Dense MLP: within-system paired Wilcoxon tests for multibasin cells and exact sign tests across systems for Dysts cells (*, $p < 0.05$). Dysts LISTA-SB is an architecture-specific two-loop/sign-split ablation at the same training budget. Standalone controls use three configured seeds and independently generated evaluation trajectories, so they are unmatched descriptive references rather than formal comparisons; only finite seed values enter their per-system summaries. Classical DMD/EDMD require each contributing start to have a finite mean through H , whereas KAE and mixture-local rows permit finite-prefix contribution. **Bold** marks the best KAE entry. For Dysts Sakarya, DMD, polynomial EDMD, and RBF-dictionary EDMD have one finite seed; the k -means H4000 cell is finite for 9/10 systems.

Model	Multibasin, 15 systems			Dysts $dt \times 30$, 10 systems		
	H100	H500	H1000	H100	H2000	H4000
LISTA	0.0413*	0.154*	0.176*	2.65×10^{-4} *	0.0999	0.465*
LISTA-BD	0.0414*	0.126*	0.147*	3.69×10^{-4}	0.122*	0.492*
LISTA-SB	0.0387 *	0.125*	0.150*	5.62×10^{-4}	0.182	0.744
Sparse MLP, BD	0.0477*	0.150*	0.178*	5.06×10^{-4}	0.130	0.441 *
Sparse MLP (Fathi et al., 2024)	0.0400*	0.0955 *	0.109 *	2.38×10^{-4}*	0.122*	0.506
Dense MLP (Lusch et al., 2018) [baseline]	0.832	2.68	2.91	0.00125	0.228	0.767
DMD	2.58	2.14	1.70	2.85	4.19	3.83
Polynomial EDMD	0.864	0.968	0.887	2.32	3.93	3.61
RBF-dictionary EDMD	0.660	0.808	0.751	0.890	2.84	2.91
k -means local linear	1.40	3.04	138	2.29	1.5×10^{150}	3.2×10^{140}
GMM local linear	1.65	3.57	3.93	10.6	7.8×10^{18}	4.5×10^{44}
Soft-gated GMM local linear	1.66	3.71	4.07	5.77	3.0×10^{13}	4.1×10^{34}



(a) Dropping the largest active coordinates.



(b) Moving active values to inactive coordinates.

Figure 1: Descriptive support-coordinate interventions for one Local-linear gates LISTA checkpoint (seed 0). **(a)** The standard rollout is compared with cumulative dropping of the largest active coordinates of the initial sparse code; bands are bootstrap 95% intervals over 100 starts. **(b)** Active values are reassigned to random inactive coordinates; the band is the interquartile range of the pooled 2,000 dependent point-shuffle outcomes (100 starts times 20 shuffles) at each horizon. No confirmatory test is attached.

at the stored observation cadence. For a start i and step t , the raw error is the squared Euclidean state error, $e_{i,t} = \sum_{j=1}^{d_x} (\hat{x}_{i,t,j} - x_{i,t,j})^2$. For the KAE rows, reported MSE through horizon H averages this state-summed error over steps $t = 1, \dots, H$ within each start and then over starts. Nonfinite step errors are omitted; a KAE rollout with a finite prefix can therefore contribute even if it diverges before H . The unmatched standalone reducers are qualified in Appendix A.

The results in table 1 show lower aggregate MSE for every sparse KAE row than for Dense MLP at the displayed horizons on both benchmarks. All controlled-multibasin sparse rows carry corrected significance annotations at all three horizons. The Dysts evidence is more heterogeneous: LISTA is significant at H100 and H4000; LISTA-BD at H2000 and H4000; Sparse MLP-BD only at H4000; Sparse MLP at H100 and H2000; and the architecture-specific LISTA-SB ablation at none of the displayed horizons. At H1000 on the controlled benchmark, Dense MLP has roughly 17–27 times the MSE of the sparse rows. The unmatched standalone controls show that conventional state-space and local-linear fits do not reproduce the best sparse-KAE errors in this packet, but the three-seed, independently generated trajectories preclude a matched causal attribution. The scale-normalized Dysts summary in table 14 provides an aggregation-sensitivity view.

Representative support-coordinate intervention. As a scoped mechanism diagnostic, we perturb the initial active set of one Local-linear gates LISTA checkpoint at seed 0 and evaluate 21-step forecasts. The evaluator first restricts candidates to the tie-inclusive high center-margin slice whose native label remains equal to its initial label throughout the 21-step true future. It balances the retained 100 starts across the three native labels (34/33/33); coordinate dropping and random support use exactly the same starts. The interventions are:

1. **Coordinate dropping:** Force the coordinates of z with the $k \in \{1, 2, 3, 5, 10\}$ largest magnitude coefficients to zero before forecasting.
2. **Random support:** the active coefficient values are preserved, but reassigned to randomly chosen inactive latent coordinates.

At $H = 21$, the standard rollout has mean accumulated MSE 0.0158, compared with 0.508/1.37/2.08/3.26/8.51 after dropping the top 1/2/3/5/10 coordinates; random support shuffling is also destructive in [figure 1](#) and [section E](#). This is descriptive evidence that this checkpoint uses coordinate identity. Because it covers one system, model, and seed and has no confirmatory test, it does not establish a general causal effect across architectures or systems.

4.2 Support-basin alignment experiments

This post-hoc experiment examines whether sparse support families fit after training are associated with evaluation-only native or proxy basin labels. For the two gated environments, the evaluator uses native labels and centers. For the 13 catalog environments, it rolls the 128 trajectory endpoints forward 5,000 steps, clusters the converged endpoints into the known benchmark count by deterministic k -means, and labels every state by its nearest estimated center. Fourteen systems realize at least two observed labels; the Triple-well Duffing proxy assigns all 16,512 states to one estimated center. Native and proxy labels and centers do not enter model training or support-family formation. The known count enters structured-transition block sizing as disclosed above, but its use here is post-hoc center estimation rather than a feature in the learned support or route.

For a state x , let $d_1(x)$ and $d_2(x)$ be its distances to the nearest and second-nearest native or estimated center. We call $d_2(x) - d_1(x)$ the *center-margin proxy*. A large value indicates separation under this nearest-center construction; it need not equal distance to a true basin boundary or separatrix, especially when basin geometry is not Voronoi. The diagnostic first fits $F_{\text{abs}}^{\text{align}}$ using the absolute threshold 10^{-3} and Jaccard threshold 0.50 over all states in each evaluation trajectory collection. It then retains states at or above the within-label 75th percentile of center margin, including all ties, and scores the already fitted partition against label B . This threshold does not resample labels to equal sizes and does not guarantee that exactly 25% of each label is retained. Conditional entropy $H(B | F_{\text{abs}}^{\text{align}})$, computed with natural logarithms and reported in nats, is the main metric; the number of family IDs observed on the high-margin slice is a descriptive companion. Most system–seed slices contain 4,129–7,167 states. Triple-well Duffing retains all 16,512 because its center margins tie, but its one-label proxy makes $H(B | F) = 0$ for every model and therefore cannot measure alignment. Primary point estimates and tests use the 14 systems with at least two observed labels; all raw Duffing rows remain in the evidence packet and an all-15-system sensitivity sidecar. Because the scored states belong to the same evaluation trajectory collection used both to estimate catalog-system centers and to fit the partition, this measures transductive alignment to native or proxy labels, not generalization of a frozen family codebook.

LISTA families align with evaluation labels on high center-margin states. In [table 2](#), LISTA has the lowest primary transductive conditional entropy, 0.138 nats, compared with 1.37

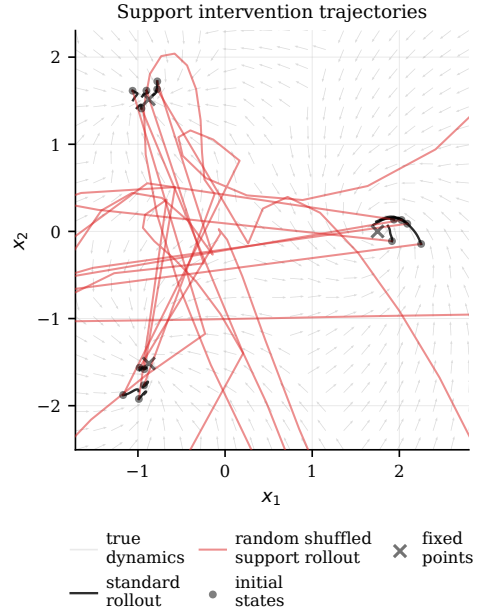


Figure 2: Randomly shuffled support trajectories (red) and standard rollouts (black) for the same representative checkpoint.

Table 2: Transductive basin-support diagnostics on the 14 controlled systems whose frozen evaluation assignment realizes at least two labels. F_{abs} is fitted over all states in each evaluation trajectory collection with absolute threshold 10^{-3} and Jaccard threshold 0.50, then scored at or above the within-label 75th center-margin percentile, retaining ties. Labels and centers are native for two gated systems and nearest-center proxies from deterministic endpoint clustering at the known benchmark count for the catalog systems. They define and score the subset but do not enter family formation or equalize label counts. $H(B | F_{\text{abs}})$ uses natural logarithms and is reported in nats. $|F_{\text{abs}}^{\text{obs}}|$ counts family IDs observed on the scored slice and uses ordinary seed means. Triple-well Duffing’s proxy realizes one label, so its identically zero entropy is excluded from primary aggregates; its raw rows and all-15-system sensitivity are retained. Superscripts mark Wilcoxon/Holm tests against Dense MLP (*, $p < 0.05$); all displayed stars pass on all 14/14 eligible systems. **Bold** marks the best directional entry.

Model	Tie-inclusive high-margin slice, 14 systems	
	$H(B F_{\text{abs}})$ [nats] ↓	$ F_{\text{abs}}^{\text{obs}} $
LISTA	0.138*	3.6
LISTA-BD	0.173*	3.5
LISTA-SB	0.168*	3.5
Sparse MLP, BD	0.386*	3.0
Sparse MLP	0.244*	3.3
Dense MLP	1.37 [baseline]	1.0 [baseline]

nats for Dense MLP. The other sparse rows lie between these values. The scored high-margin slice contains 3.6 LISTA family IDs on average, compared with a mean/median of 4.29/4 represented labels across the 14 systems, whereas Dense MLP exposes one family ID on average. These are ordinary seed means of observed slice counts, not total partition sizes. All-15-system sensitivity values are 0.129 and 1.28 nats for LISTA and Dense MLP, respectively; including the uninformative zero therefore reduces both estimates. Alignment tests pass after correction on all 14/14 eligible systems. This is evidence of post-hoc alignment to this evaluation label construction; it does not show that the center margin measures true boundary distance, that a frozen family map discovers new basins out of sample, or that every family corresponds one-to-one with a basin.

4.3 Support-family local linearizations

We next test a separate, controlled routing protocol. The staged model and global- K baseline use the same LISTA architecture and total 200,000-step budget. Stage one trains the encoder, decoder, and shared K for 100,000 steps. We then freeze all three and fit $F_{\text{abs}}^{\text{route}}$ on a nominal 512-row packet from the training distribution. The source implementation generated one batch of 256 unique trajectories and repeated it exactly, so this packet contains 256, not 512, unique trajectories. Each trajectory has 192 transitions and 193 states. The greedy 10^{-3} -threshold, Jaccard-0.40 family merge sees all 193 state supports, including terminal states; map counts, source centers, and direct mask keys then use the first 192 source states. This gives 98,816 nominal supports for clustering, 98,304 nominal source transitions, and 49,152 unique source transitions. Families with at least one source transition receive affine maps trained for another 100,000 steps.

For support family f , let $\bar{z}_f$ be the mean source latent assigned to that family. The route-local

Table 3: Staged $F_{\text{abs}}^{\text{route}}$ -routed local affine forecasting on the controlled benchmark. The same-architecture, same-budget global- K LISTA baseline has $d_z = 256$, but checkpoint selection is asymmetric: the staged selector overlaps the first 32/100 reported starts, whereas the global selector uses distinct starts. Both rows also select cadence on the reported starts over the same fixed grid. Pair counts are descriptive nested system–seed outcomes; the last column counts pairs won by the routed model at H100, H500, and H1000 simultaneously. Lower MSEs and ratios are better.

Metric	H100	H500	H1000	All three horizons
Paired local wins	199/225	192/225	189/225	180/225
Mean MSE, $F_{\text{abs}}^{\text{route}}$ -local/global	0.0128/0.0229	0.0347/0.0623	0.0403/0.0706	–
Median local/global MSE ratio	0.355	0.312	0.328	–
Geometric local/global MSE ratio	0.252	0.180	0.174	–

update is

$$T_f(z) = d_f + K_f(z - \bar{z}_f), \quad (13)$$

where K_f is initialized from the frozen global matrix and the learned intercept d_f is initialized to $K\bar{z}_f$. The runtime representative of each merged family is its modal source-state support mask. Before every latent transition, the model recomputes the current latent’s absolute support and routes that step against the frozen representatives; an unassigned route falls back to the frozen global K . Scheduled periodic events instead decode and re-encode the predicted state, refreshing the latent before the next step’s route. Thus routing occurs every latent step even when re-encoding is less frequent. Neither construction nor rollout uses basin labels, counts, attractor identities, or trajectory assignments. For each horizon, both staged and global rows are reported at their evaluation-set oracle cadence from $m \in \{1, 2, 5, 10, 20, 25, 50, 100\}$.

Checkpoint selection is not matched. The staged selector scores exactly 200 candidates—steps 100,500,101,000, . . . , 199,500, plus 199,999—on 32 fixed starts at seed offset 12,345. For each candidate, it minimizes the finite-prefix, state-summed squared error independently across the cadence grid at H100, H500, and H1000, averages those three minima, and replaces the incumbent only on a strict improvement. Final reporting uses 100 starts at the same seed offset, so the selector starts are exactly the first 32 reported starts (32% overlap). By contrast, the global baseline uses its standard 16-start, seed-offset-999,999, every-step-re-encoded H200 final-error selector, which has no overlap with the reported starts. Thus architecture and optimization budget are shared, but the staged comparison is descriptive and optimistic rather than a clean held-out checkpoint comparison.

Across the 15 controlled systems and 15 seeds, the routed model has lower MSE on 199/225, 192/225, and 189/225 system–seed pairs at H100, H500, and H1000; median local/global ratios are 0.355, 0.312, and 0.328. These 225 pairs are nested within 15 systems and are reported descriptively, without treating them as independent units or attaching confirmatory inference. They show lower observed error for this exact staged artifact, but selector overlap and asymmetry preclude a clean causal or held-out routing comparison.

5 Discussion

The evidence supports three bounded claims. Sparse-latent KAEs forecast better than the reported Dense MLP KAE rows on the controlled benchmark and retain an aggregate advantage on the Dysts stress test, with the row-specific significance pattern reported above. On high center-margin controlled-benchmark states, LISTA families have low transductive conditional entropy with evaluation-only native or nearest-center-proxy labels. In a separate staged artifact,

training-distribution support families route local affine maps and have descriptively lower controlled-forecasting error than a same-budget global- K LISTA model, subject to the selector leakage and asymmetry above. The one-checkpoint coordinate intervention is consistent with coordinate identity mattering for that predictor, but it is ancillary descriptive evidence rather than a fourth general claim.

These results motivate a hybrid interpretation in which coefficient values carry continuous predictive coordinates and active-coordinate identities provide an inspectable route key. We do not claim an exact global finite-dimensional linearization, a one-to-one support-basin theorem, or supervised-quality basin recovery. The theory instead motivates caution about one continuous global representation, while the experiments test whether sparsity provides a useful empirical alternative. The staged result is likewise specific to the fitted support-family partition and affine-map recipe; it does not show that sparse routing is the only useful local-model selector.

Several protocol limitations constrain the conclusions. First, all reported best-periodic values choose cadence separately at each horizon on the same test trajectories; validation-frozen cadence selection is needed for deployment-style estimates. The MSE reducer also omits nonfinite steps, so finite prefixes can make divergent rollouts appear better than a strict full-horizon failure rule. Every ordinary controlled row and both stages of the staged model also cycle only eight fixed training batches per seed because the maintained child-seed streams are not advanced, substantially limiting training-data diversity. Second, the alignment partition is fitted over the same evaluation trajectory collection that contains the high center-margin states used for scoring. For 13 systems, the same collection also supplies the endpoints used to estimate centers, and the known benchmark count enters that evaluation-only construction. The resulting nearest-center labels and $d_2 - d_1$ margin are proxies and need not recover true basin geometry or boundary distance. This is transductive evidence on two-dimensional procedural systems, not out-of-sample basin discovery, behavior near separatrices, or evidence for high-dimensional physical systems. The activation and Jaccard thresholds differ by experiment (0.50 for alignment and 0.40 for staged routing), and sensitivity to those fixed choices remains a limitation.

Third, the 225 staged system-seed pairs are nested within 15 systems and receive no confirmatory system-level inference. Its route-fit packet contains only 256 unique trajectories despite 512 configured rows, and its checkpoint selector reuses 32% of the reported starts while the global baseline uses a different selector. The result should be rerun from a newly versioned 512-unique route fit with disjoint, common checkpoint-validation starts and a validation-frozen cadence before making a general routing claim. Cross-family attribution is also limited because LISTA decoders are atom-normalized whereas MLP decoders are not; Dysts LISTA-SB uses a different loop/output architecture from LISTA and LISTA-BD; and the structured-transition rows use a known controlled-benchmark basin count or a hand-set Dysts lobe/scroll/equilibrium count solely to size their blocks. Finally, standalone controls use three seeds and independently generated trajectories, and the coordinate intervention uses only one Local-linear gates LISTA checkpoint at seed 0. These comparisons remain descriptive.

The immediate priorities are therefore to freeze period and family choices on disjoint validation data, score them on untouched test trajectories with strict finiteness accounting, rerun the staged comparison under one common selector and a newly versioned 512-unique route fit with system-level uncertainty, and test basin-support alignment beyond the transductive two-dimensional high center-margin slice with proxy labels.

A Additional experimental details

This appendix records the paper-facing protocol. Unstructured LISTA, Sparse MLP, Dense MLP, support-family construction, and staged routing are trained or constructed without basin labels, basin counts, or trajectory-to-basin assignments. For post-hoc alignment evaluation, the two gated systems provide native labels and centers; the other 13 systems use endpoint-estimated centers and nearest-center proxy labels, with the known benchmark count setting the number of estimated centers. Separately, the block-diagonal and soft-block architecture diagnostics use the known controlled count to set the number of transition blocks, while the Dysts rows use a hand-specified lobe, scroll, or equilibrium count. No label or count is a feature in support-family formation or routing, and structured rows receive no trajectory label.

Benchmarks. The controlled benchmark contains 15 two-dimensional multibasin systems, with equations and retained metadata in [section B](#). Unstructured rows and runtime prediction observe stored states, not vector fields, integration substeps, native basin annotations, estimated centers, proxy labels, or benchmark counts; the disclosed structured-transition rows use a count only to size their parameterization. The external forecasting stress test uses the fixed retained subset of 10 three-dimensional Dysts $dt \times 30$ systems in [section C](#). Dysts observations are generated at 30 times each system’s native integration interval and converted to standardized coordinates using that pinned system’s metadata mean and standard deviation.

Model rows and decoder normalization. All six KAE rows have $d_z = 256$. Encoder pre-code or MLP hidden widths are 64, 64. Sparse MLP and LISTA pre-code layers use biased ReLU activations, whereas Dense MLP uses tanh; all decoders are linear. LISTA, LISTA-BD, and LISTA-SB use learned shrinkage; Sparse MLP and Sparse MLP-BD use an L_1 latent penalty; Dense MLP removes the sparse output gate and sets that penalty to zero. BD denotes a block-diagonal K ; SB denotes a dense K with an off-block penalty. The reported LISTA models decode with a normalized linear dictionary. The sparse and dense MLP rows use ordinary learned linear decoders without atom normalization, so decoder scaling is not controlled across encoder families.

The controlled LISTA rows use threshold scale $\alpha = 0.15$, two refinement loops, and a sign-split output that maps a signed 128-dimensional base code to $d_z = 256$ by concatenating its positive and negative parts. The shrinkage threshold is $\alpha/\hat{L}_D$, where initialization estimates $\hat{L}_D$ as 1.05 times a ten-step power-iteration estimate of the initialized dictionary Gram norm. On Dysts, LISTA and LISTA-BD use $\alpha = 0.15$, one loop, and a ReLU output directly in 256 dimensions. Dysts LISTA-SB is an architecture-specific ablation at the same training budget: it uses two loops and a sign-split output with soft-block regularization. Its run packet covered 12 systems; all paper aggregates use the fixed retained 10-system subset.

Controlled multibasin training. Every controlled row uses seeds $0, \dots, 14$, rollout-window length $L = 8$, 200,000 optimization steps, minibatches of 256, and AdamW. Encoder and decoder learning rates are 5×10^{-5} , the transition learning rate is 5×10^{-6} , and non-transition weight decay is 10^{-4} . The objective weights are $\lambda_{\text{pred}} = 1$, $\lambda_{\text{rec}} = 0.03$, $\lambda_{\text{lin}} = 1$, and $\lambda_{\text{sp}} = 3 \times 10^{-3}$ for sparse rows; Dense MLP uses $\lambda_{\text{sp}} = 0$. Soft-block rows add an off-block L_1 penalty with weight 10^{-4} .

All controlled rows use the system-specific observation intervals and base reset boxes in [table 5](#), followed by the same label-free perturbation-sensitive, high-transient training wrapper. Let s be the coordinatewise span of the base reset box. For model seed r , candidate j uses base-reset seed $r + 2,000,000 + j$. A single float32 noise stream seeded $r + 3,000,000$ is consumed in candidate-order

chunks of 128. It supplies four Gaussian perturbations per candidate with coordinatewise standard deviation $0.04s$; these are box-clipped and rolled for 32 stored steps. The score is their mean endpoint separation from the unperturbed rollout, normalized by $\|0.04s\|_2$, plus 0.5 times the mean step-24-to-step-32 motion over all five rollouts, normalized by $\|s\|_2$. The top 1,024 of 4,096 candidates form the hard pool. Each training reset chooses that pool with probability 0.5; a selected point receives box-clipped Gaussian jitter with coordinatewise standard deviation $0.01s$, while the other branch uses a base reset. This rule uses no basin labels or boundary geometry and should not be interpreted as sampling measured basin boundaries.

The maintained loader cycles eight fixed batches of 256 length-eight windows per model seed because its eight child-seed streams are not advanced between uses. This fixed-data artifact affects all six ordinary controlled KAE rows and both training stages of the staged model; its global- K comparator is the ordinary LISTA row. Stream $b \in \{0, \dots, 7\}$ supplies child reset seeds $r + 256b + i$, $i = 0, \dots, 255$, to the wrapped reset rule above.

Dysts training and caches. All retained Dysts rows use seeds $0, \dots, 14$, $d_z = 256$, batch size 256, $L = 10$, and 100,000 optimization steps. Deterministic caches contain 200 trajectories of 30,000 observations after a 2,000-step warm-up. The first trajectory starts from the Dysts default initial condition; the others use Gaussian perturbations at 0.2 times the coordinate-wise Dysts standard deviation. Train, validation, and test caches have separate deterministic namespaces. LISTA rows use learning rates 5×10^{-5} for encoder/decoder parameters and 5×10^{-6} for K . MLP rows use 10^{-4} and 10^{-5} , respectively. Sparse rows use $\lambda_{\text{sp}} = 6 \times 10^{-3}$; Dense MLP uses zero.

Checkpoint selection. For the ordinary single-stage KAE runs, including the global- K LISTA baseline used in the staged table, every 500 optimization steps and the final step are candidates. Each candidate is rolled out for 200 stored steps from 16 fixed starts generated at seed offset 999,999, using every-step re-encoding. The checkpoint with the lowest mean final-step Euclidean state error is retained. These starts are distinct from the 100 reported evaluation starts at seed offset 12,345. The staged selector is different and is specified below. Tables aggregate completed seeds and do not choose the best seed.

Forecasting metric and cadence selection. For start i , step t , and state dimension j , the raw step error is $e_{i,t} = \sum_j (\hat{x}_{i,t,j} - x_{i,t,j})^2$. For KAE and local-linear mixture rows, the reported MSE through H is the mean of finite $e_{i,t}$ over $t = 1, \dots, H$ within each start and then over starts. Consequently, a rollout with a finite prefix can contribute even if it becomes nonfinite before H ; strict full-horizon finite coverage is not their ranking rule. The classical DMD, polynomial EDMD, and RBF-dictionary EDMD evaluator instead takes an ordinary mean through H within each start and then omits starts whose resulting mean is nonfinite, so those descriptive rows require full finite-step coverage through H for a start to contribute.

Periodic rollout decodes and re-encodes only the model’s own prediction. For model seed r , controlled forecasting uses 100 base-reset starts with child seeds $r + 12,345 + i$, $i = 0, \dots, 99$; the training-only hard-pool wrapper is not used. The alignment collection analogously uses base-reset seeds $42 + i$, $i = 0, \dots, 127$, and is shared across model seeds. Controlled forecasting uses cadence grid $m \in \{10, 25, 50, 100\}$; Dysts uses 100 held-out test-cache windows and $m \in \{10, 25, 50, 100, 150, 200\}$. For each model, system, seed, and horizon, the reported best-periodic value is the minimum on those same evaluation trajectories over the fixed grid. Thus cadence is selected by an evaluation-set oracle rather than frozen from validation. The staged table uses the wider grid given below.

Standalone controls. DMD, polynomial EDMD, RBF-dictionary EDMD, and the local-linear mixture controls configure seeds 0, 1, 2 and use independently generated trajectories rather than the KAE rows’ matched test samples. The controlled manifest uses 256 trajectories of length 1000, with 60% assigned to fitting. The Dysts manifest uses 200 trajectories of length 4000, with 50% assigned to fitting, standardized coordinates, and the same $dt \times 30$ observation cadence. All fits use ridge coefficient 10^{-6} . Polynomial EDMD uses degree 3; RBF-dictionary EDMD uses 128 centers with data-derived gamma and the implementation default as fallback; k -means, GMM, and soft-gated GMM local-linear controls use four components. Evaluation horizons match the corresponding columns of [table 1](#). Per-system summaries use the available finite seeds: on Dysts Sakarya, DMD, polynomial EDMD, and RBF-dictionary EDMD have one finite seed at the three displayed horizons; the Dysts k -means H4000 aggregate has finite coverage on 9/10 systems. Exact finite-seed and finite-system counts are retained in the table sidecars. These rows are descriptive references only and do not support paired tests or a causal conclusion about sparsity.

Transductive support–basin alignment. At evaluation seed 42, the evaluator generates 128 trajectories with 128 transitions and therefore 129 states per row, or 16,512 states before subset selection. It constructs $S_{\text{abs}}(x) = \{i : |z_i| > 10^{-3}\}$ for all of those states and greedily merges their masks at Jaccard threshold 0.50 to form $F_{\text{abs}}^{\text{align}}$. The Local-linear gates and Transfer-gated local-linear environments provide native labels and centers. Each of the 13 catalog environments provides neither: the evaluator advances the 128 trajectory endpoints for 5,000 additional steps, uses the known manifest basin count in a deterministic k -means fit (first endpoint followed by farthest-point initialization, then at most 25 Lloyd updates), and labels every state by its nearest estimated center.

For every state, the evaluator computes the distance gap $d_2 - d_1$ between its second-nearest and nearest native or estimated center. Within each observed evaluation label, it retains every state at or above that label’s 75th margin percentile, including ties, and scores $H(B \mid F_{\text{abs}}^{\text{align}})$, using natural logarithms and reporting nats. This operation does not equalize label sample counts or force an exact 25% retention rate. Most system–seed subsets contain 4,129–7,167 states. Tied margins make Triple-well Duffing retain all 16,512, and its proxy assigns every state to one estimated center. The resulting $H(B \mid F) = 0$ for all models is uninformative, so the primary alignment aggregate uses the 14 systems with at least two observed labels. Raw Duffing rows remain frozen and an all-15-system sensitivity is retained. A large margin means that the nearest-center assignment is separated from its runner-up; it need not measure distance to a true basin boundary or separatrix. Labels, centers, and the known catalog-system count define and score this subset but do not enter family formation. The result is post-hoc and transductive because the scored states are contained in the same evaluation trajectory collection used to fit the partition and, for catalog systems, to estimate centers; it is not evaluation of a codebook or label rule fitted on disjoint data. The companion family count is the number of fitted family IDs observed on the scored subset, not the total partition size over all 16,512 states.

Representative coordinate intervention. The intervention uses one Local-linear gates LISTA checkpoint at seed 0. It either drops the $k \in \{1, 2, 3, 5, 10\}$ largest-magnitude entries of z_0 or moves active values to randomly selected inactive coordinates, then runs the ordinary transition and decoder. Candidate starts must lie in the tie-inclusive per-native-label high center-margin slice and retain their initial native label throughout the true 21-step future. Sampling balances the 100 starts over sorted native labels, giving 34/33/33; coordinate dropping and the 20-shuffle random-support condition use exactly the same starts. The margin is the same nearest-center separation proxy, not

a measured distance to a basin boundary. This intervention is descriptive and has no confirmatory significance test.

Staged support-family local forecasting. The staged and global- K LISTA models use the same architecture and 200,000-step total budget. Stage one trains E_θ , D_ϕ , and a shared K for 100,000 steps. The staged model then freezes them and fits $F_{\text{abs}}^{\text{route}}$ at seed offset 271,828. Its nominal 512-row packet is two bitwise-identical copies of one 256-trajectory batch from the training distribution. A row contains 192 transitions and 193 states. Family formation at absolute threshold 10^{-3} and Jaccard threshold 0.40 consumes all 193 supports per row, so terminal supports affect frequency ordering and the greedy partition. Map counts, source centers, and direct mask-to-family keys use only the first 192 source states. The corresponding nominal counts are 98,816 clustered supports and 98,304 source transitions, of which 49,152 source transitions are unique. A future fit with 512 unique trajectories is a new protocol and requires new checkpoints.

After the greedy merge, the frozen runtime representative for each family is the modal support among its source states, with the historical first-seen tie break; it need not equal the initial frequency-ordered mask that opened the family. Families need at least one source transition. The model trains one affine map $T_f(z) = d_f + K_f(z - \bar{z}_f)$ per retained family for 100,000 further steps. Each K_f starts from the frozen global K , while d_f is a learned intercept initialized to $K\bar{z}_f$.

During rollout, the staged wrapper computes the current latent’s absolute support and selects a family before every latent transition. Unassigned masks use global K . Independently, each scheduled periodic refresh decodes and re-encodes the current prediction; that refreshed latent is routed at the following transition. Period m therefore controls latent refresh, not route lookup: route lookup occurs on all steps. Neither fitting nor routing uses basin labels or counts. Both staged and global rows are reported at their evaluation-set oracle cadence for each horizon from $m \in \{1, 2, 5, 10, 20, 25, 50, 100\}$.

The staged checkpoint selector evaluates exactly 200 candidates: steps 100,500 through 199,500 every 500 steps, plus step 199,999. Its 32 fixed starts use seed offset 12,345. For every candidate and cadence, it computes state-summed squared error, averages each start’s finite prefix through H100, H500, or H1000, and then averages the finite per-start means. It selects the best cadence independently at each horizon, takes the arithmetic mean of the three minima, and saves only on a strict decrease. Final reporting uses 100 starts generated at the same offset; therefore the selector starts are exactly the first 32 reported starts, a 32% overlap. The global baseline instead uses the standard 16-start offset-999,999, every-step H200 final-error selector described above. Hence the comparison matches architecture and total budget but not checkpoint selection, and is descriptive and optimistic. The 225 system–seed pairs are also nested within 15 systems; win counts and ratios have no confirmatory p -value.

Aggregation and inference. Forecasting and alignment entropy point estimates use SciPy’s `trim_mean` with proportion 0.25 within each system, followed by an arithmetic mean across systems. Thus $\lfloor n/4 \rfloor$ finite values are discarded from each tail and a complete 15-seed cell averages its central nine. Controlled forecasting uses all 15 systems; primary alignment uses the 14 systems with at least two observed labels and retains an all-15 sensitivity. The descriptive family-count column instead uses ordinary seed means within system. Controlled forecasting and alignment stars use paired seed-level Wilcoxon tests within system with Holm correction across eligible systems. Dysts stars use each system as the independent unit after the same within-system trimmed mean and an exact sign test with Holm correction. The staged and intervention results are descriptive. Full definitions are in [section F](#).

Table 4: The 15 multibasin benchmark systems. The manifest count sets the number of estimated evaluation centers for catalog systems and the number of transition blocks for structured-transition diagnostic rows.

Display name	Dynamical family	Manifest count (eval./blocks)
Local-linear gates	piecewise local-linear gates	3
Transfer-gated local-linear	piecewise transfer gates	3
Arrested spiral	spiral capture with Gaussian traps	5
Asymmetric three-well	Gaussian wells with independent rotation	3
High-cross three-well	Gaussian wells with stronger rotation	3
Hexagonal six-well	Gaussian wells with independent rotation	6
Octagonal eight-well	Gaussian wells with independent rotation	8
Pentagonal five-well	Gaussian wells with independent rotation	5
Square four-well	Gaussian wells with independent rotation	4
Triple-well Duffing	triple-well Duffing oscillator	3
SNIC multi-attractor	polar SNIC-like phase dynamics	3
Transition-routes four-well	route-augmented Gaussian wells	4
Depth-gradient four-well	Gaussian wells with depth gradient	4
Diamond four-well	Gaussian wells on a diamond layout	4
L-shaped five-well	Gaussian wells on an L-shaped layout	5

Hardware. Training used one cluster-assigned CUDA GPU per allocation with four CPU cores. The main controlled and Dysts KAE rows requested 16 GB host memory, whereas staged support-family training requested 24 GB. Jobs loaded CUDA 12.6.0. Controlled training requested 24 hours per run; packed Dysts allocations requested up to 72 hours. Cache construction and paper evaluation used CPU allocations; the retained coordinate-intervention evaluator requests 8 GB. The scripts do not pin a GPU model, so a specific accelerator model is not claimed.

B Multibasin benchmark inventory and system definitions

Table 4 lists the 15 two-dimensional multibasin systems used in this paper. Every row contributes to controlled forecasting and staged-routing results in tables 1 and 3. The primary alignment aggregate in table 2 uses the 14 systems whose frozen evaluation assignment realizes at least two labels. Triple-well Duffing’s one-label proxy rows remain in the evidence packet and in the all-15-system sensitivity sidecar.

The two gated systems expose native labels and centers for evaluation. The remaining 13 catalog systems expose neither; their alignment evaluator uses the known benchmark count to estimate centers from long-rolled endpoints and construct nearest-center proxy labels. Basin counts are also used to set the fixed architecture group count for structured-transition ablations, as described in section A. No native label, proxy label, center, or count is supplied as a runtime feature to support extraction, support-family construction, staged routing, or rollout prediction; structured-transition rows retain the disclosed count-sized parameterization.

For every system except Triple-well Duffing, the frozen seed-42 evaluation assignment realizes the full manifest count shown above. The Duffing endpoint clustering instead assigns all 16,512 evaluation states to one estimated center. Its conditional entropy is therefore zero for every model and cannot measure regime alignment; this evaluation-proxy failure does not remove the system from forecasting or staged-routing experiments.

Table 5: Stored observation intervals and base reset laws. Each coordinate is sampled independently and uniformly over the listed box; one stored step is one fourth-order Runge–Kutta update.

System(s)	Δt	Base reset box
Local-linear gates	0.04	$[-2.6, 2.6]^2$
Transfer-gated local-linear	0.04	$[-2.8, 2.8]^2$
Arrested spiral	0.02	$[-3, 3]^2$
Asymmetric three-well	0.03	$[-3, 3] \times [-2.5, 3]$
High-cross three-well; Hexagonal six-well; Pentagonal five-well; Square four-well	0.03	$[-3.3, 3.3]^2$
Octagonal eight-well	0.03	$[-4, 4]^2$
Triple-well Duffing; SNIC multi-attractor	0.02	$[-2, 2]^2$
Transition-routes four-well	0.02	$[-3.5, 3.5]^2$
Depth-gradient four-well	0.03	$[-3, 3]^2$
Diamond four-well; L-shaped five-well	0.03	$[-3.5, 3.5]^2$

Stored-step convention. All retained systems are implemented as continuous-time vector fields $\dot{x} = f_s(x)$ and integrated with fourth-order Runge–Kutta at the system-specific intervals in [table 5](#) to produce the stored observation map $F_{\Delta t}$ used in [equation \(1\)](#). The training windows contain only stored states x_k , not f_s , integration substeps, or basin assignments.

Ground-truth vector-field visualizations. [Figure 3](#) shows the continuous-time ground-truth vector fields for all 15 retained two-dimensional multibasin systems. Streamlines show the direction of $f_s(x)$, and black crosses mark the generator attractor centers or analytic equilibrium references used for benchmark annotation. The streamline colors use a panel-local $\log_{10} \|f_s(x)\|_2$ scale to reveal structure within each system; they should not be read as comparable speed scales across panels. Detailed panels with per-system colorbars are provided in [figures 4 to 8](#).

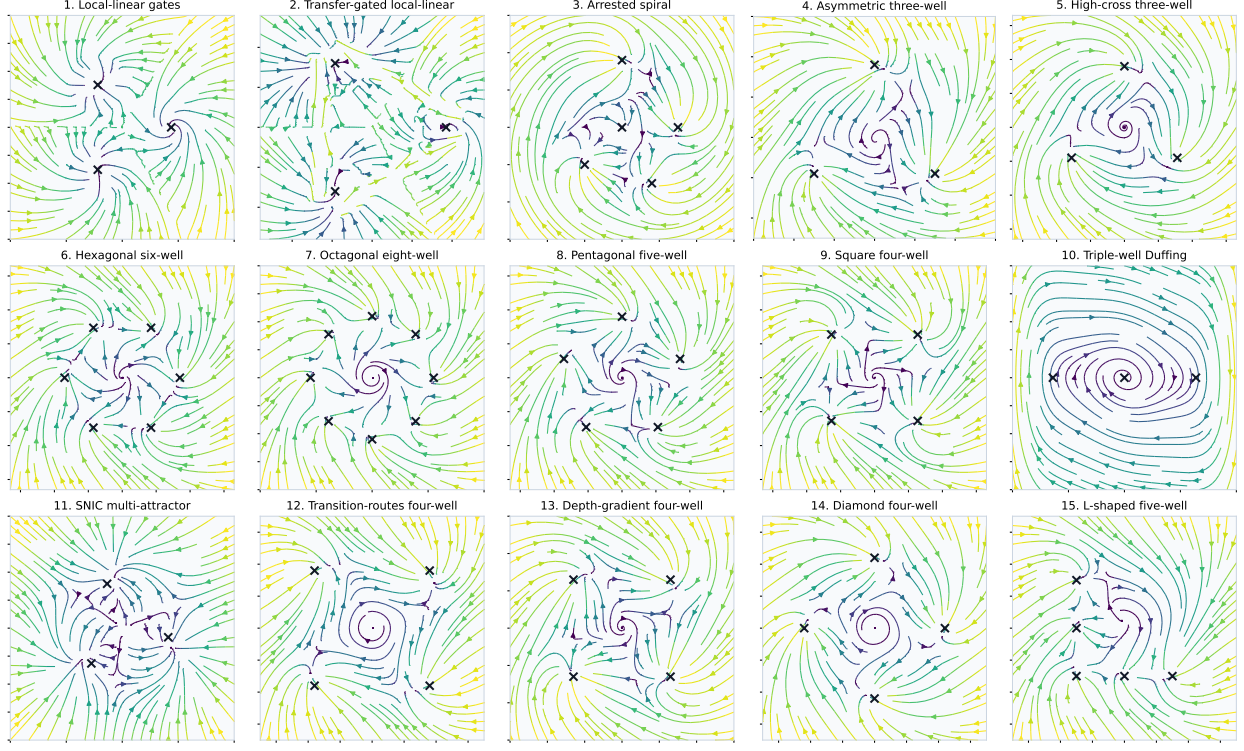


Figure 3: Ground-truth vector fields for the retained 15-system multibasin benchmark. The models are trained only on stored state trajectories; these continuous-time vector fields and generator reference markers are shown only to document the benchmark and are not provided to the learner. For the 13 catalog systems, these markers are not native alignment labels or centers; the evaluation-only proxies are estimated from long-rolled endpoints.

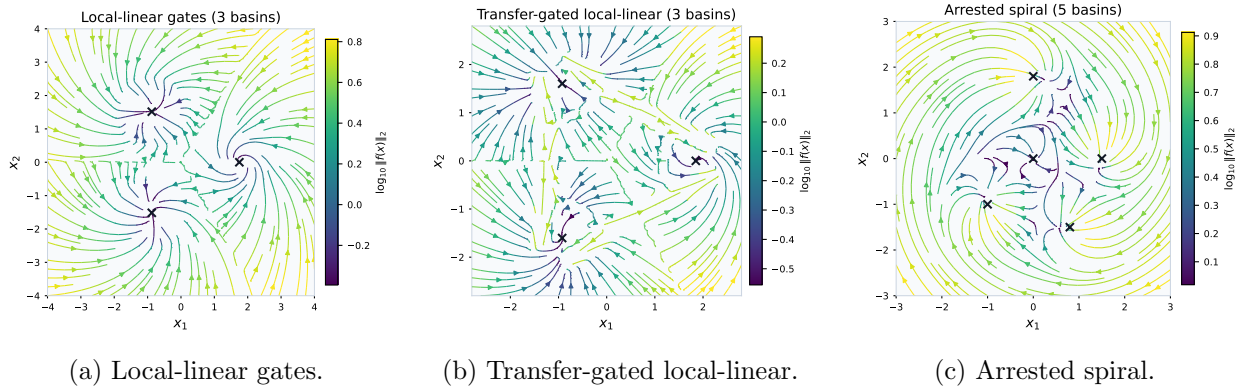


Figure 4: Detailed ground-truth vector-field panels for the first three retained multibasin systems. Colorbars report the panel-local $\log_{10} \|f_s(x)\|_2$ scale.

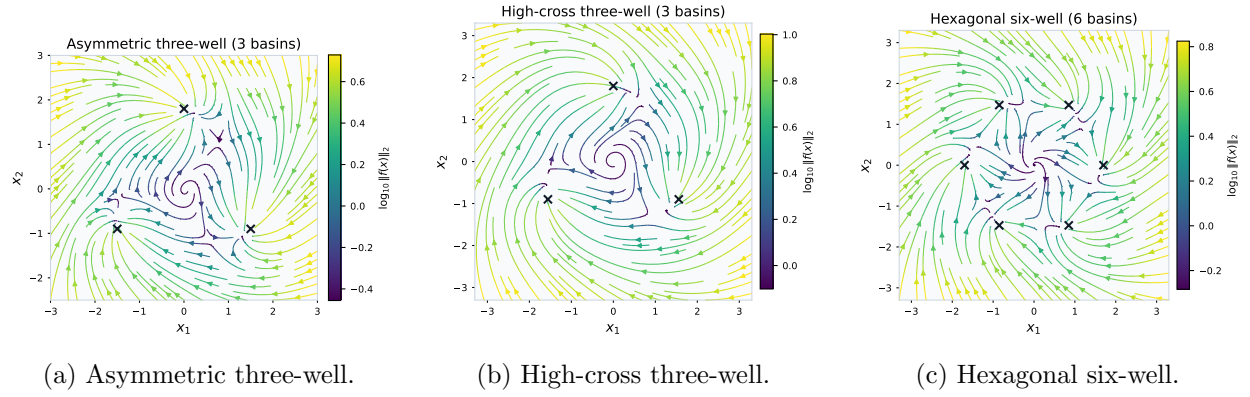


Figure 5: Detailed ground-truth vector-field panels for retained Gaussian-well systems. Colorbars report the panel-local $\log_{10} \|f_s(x)\|_2$ scale.

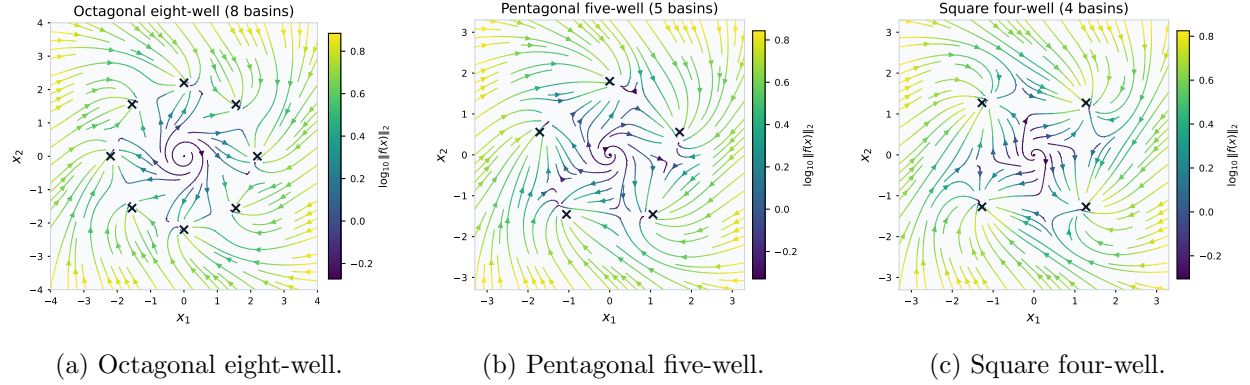


Figure 6: Detailed ground-truth vector-field panels for retained polygonal Gaussian-well systems. Colorbars report the panel-local $\log_{10} \|f_s(x)\|_2$ scale.

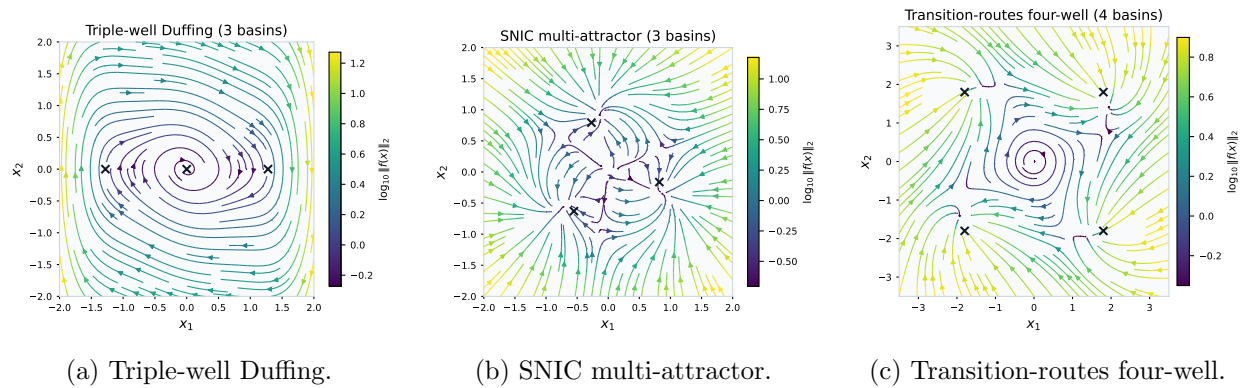


Figure 7: Detailed ground-truth vector-field panels for specialized retained systems. Colorbars report the panel-local $\log_{10} \|f_s(x)\|_2$ scale.

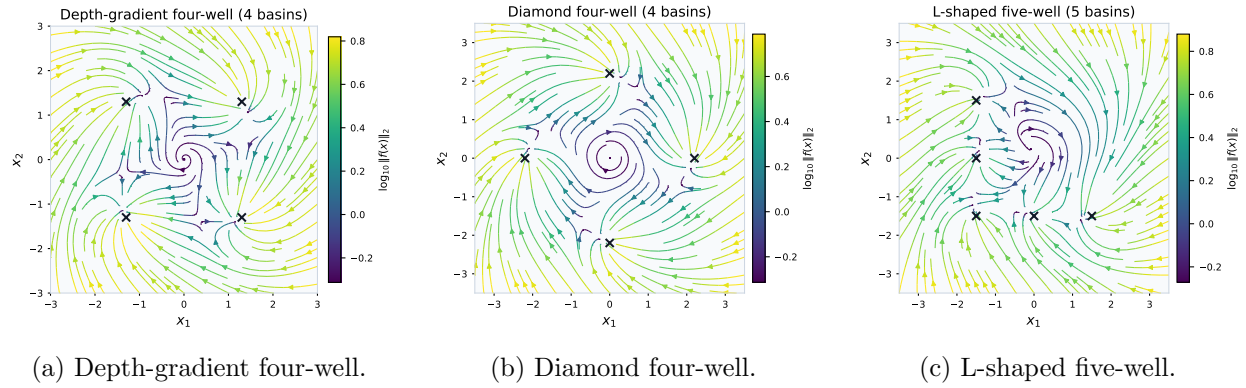


Figure 8: Detailed ground-truth vector-field panels for retained geometry variants. Colorbars report the panel-local $\log_{10} \|f_s(x)\|_2$ scale.

Table 6: Parameters for retained Gaussian-well systems. Repeated pairs in the (a_i, σ_i) column apply to every listed center.

System	$\{c_i\}$	(a_i, σ_i)	(ω, γ)
High-cross three-well	$\{p_i^{(3)}(1.8, \pi/2)\}$	(3.0, 0.5)	(2.0, 0.03)
Hexagonal six-well	$\{p_i^{(6)}(1.7, 0)\}$	(2.0, 0.5)	(1.0, 0.03)
Octagonal eight-well	$\{p_i^{(8)}(2.2, 0)\}$	(3.0, 0.5)	(0.90, 0.02)
Pentagonal five-well	$\{p_i^{(5)}(1.8, \pi/2)\}$	(2.0, 0.5)	(1.1, 0.03)
Square four-well	$\{p_i^{(4)}(1.8, \pi/4)\}$	(3.0, 0.5)	(1.0, 0.03)
Diamond four-well	$\{(0, 2.2), (2.2, 0), (0, -2.2), (-2.2, 0)\}$	(2.5, 0.5)	(1.0, 0.02)
L-shaped five-well	$\{(-1.5, 1.5), (-1.5, 0), (-1.5, -1.5), (0, -1.5), (1.5, -1.5)\}$	(2.5, 0.5)	(1.0, 0.03)
Asymmetric three-well	$\{(0, 1.8), (-1.5, -0.9), (1.5, -0.9)\}$	$\{(2.5, 0.55), (1.5, 0.4), (2.0, 0.5)\}$	(1.0, 0.03)
Depth-gradient four-well	$\{(-1.3, 1.3), (1.3, 1.3), (1.3, -1.3), (-1.3, -1.3)\}$	$\{(2.2, 0.55), (2.5, 0.5), (3.0, 0.5), (3.5, 0.5)\}$	(1.3, 0.03)

Gaussian-well family. Most retained catalog systems use a common two-dimensional Gaussian potential with an independent rotational drift. For $x = (x_1, x_2) \in \mathbb{R}^2$, let $Rx = (x_2, -x_1)$ and

$$V_s(x) = \sum_{i=1}^{M_s} -a_{s,i} \exp\left(-\frac{\|x - c_{s,i}\|_2^2}{2\sigma_{s,i}^2}\right) + \gamma_s(x_1^4 + x_2^4).$$

The implemented vector field is

$$\dot{x} = -\nabla V_s(x) + \omega_s Rx. \quad (14)$$

Define $p_i^{(M)}(r, \varphi) = r(\cos(2\pi i/M + \varphi), \sin(2\pi i/M + \varphi))$, $i = 0, \dots, M-1$. The retained Gaussian-well systems instantiate [equation \(14\)](#) with the parameters in [table 6](#).

The transition-routes system uses the same Gaussian-well form with centers

$$\{c_i\} = \{(-1.8, 1.8), (1.8, 1.8), (-1.8, -1.8), (1.8, -1.8)\},$$

$a_i = 3.0$, $\sigma_i = 0.6$, $\omega = 1.0$, and $\gamma = 0.03$, and adds a corridor-dependent rotation term:

$$\dot{x} = -\nabla V_s(x) + (1.0 + 0.3 C_{\text{route}}(x_2)) Rx, \quad C_{\text{route}}(x_2) = e^{-(x_2-1.8)^2/0.3} + e^{-(x_2+1.8)^2/0.3}. \quad (15)$$

Native gated local-linear systems. Both native gated systems place three centers on a circular layout, but with different radii and region definitions. Let $Q(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha \\ \sin \alpha & \cos \alpha \end{pmatrix}$ and $\alpha_b = 2\pi b/3$, $b \in \{0, 1, 2\}$.

For Local-linear gates, $c_b = 1.75(\cos \alpha_b, \sin \alpha_b)$. Inside the radius-1.05 core of basin b ,

$$\dot{x} = A_b(x - c_b), \quad A_b = Q(\alpha_b) \tilde{A}_b Q(\alpha_b)^\top,$$

with

$$\tilde{A}_0 = \begin{pmatrix} -0.9 & -1.2 \\ 1.2 & -0.9 \end{pmatrix}, \quad \tilde{A}_1 = \begin{pmatrix} -1.35 & 0.2 \\ -0.3 & -0.7 \end{pmatrix}, \quad \tilde{A}_2 = \begin{pmatrix} -0.7 & -0.1 \\ 0.5 & -1.2 \end{pmatrix}.$$

Outside the cores, the active sector is the nearest angular sector $s(x)$ and the shared gate matrix

$$G = \begin{pmatrix} -1.35 & -0.9 \\ 0.9 & -1.35 \end{pmatrix}$$

drives the state as $\dot{x} = G(x - c_{s(x)})$.

For Transfer-gated local-linear, $c_b = 1.85(\cos \alpha_b, \sin \alpha_b)$. The implemented radii and widths are

$$\begin{aligned} \rho_{\text{core}} &= 0.30, & \rho_{\text{source}} &= 0.80, & \rho_{\text{exit}} &= 0.60, & \beta_{\text{exit}} &= 0.72, \\ w_{\text{chan}} &= 0.22, & \delta_{\text{lane}} &= 0.28, & \rho_{\text{hand}} &= 0.45. \end{aligned}$$

The core matrices use the same rotated form with templates

$$\begin{pmatrix} -1.0 & -1.1 \\ 1.1 & -1.0 \end{pmatrix}, \quad \begin{pmatrix} -1.4 & 0.2 \\ -0.2 & -0.7 \end{pmatrix}, \quad \begin{pmatrix} -0.8 & -0.3 \\ 0.5 & -1.3 \end{pmatrix}.$$

For each ordered pair $p = (s, t)$, $s \neq t$, define

$$d_{s \rightarrow t} = \frac{c_t - c_s}{\|c_t - c_s\|_2}, \quad n_{s \rightarrow t} = (-d_{s \rightarrow t, 2}, d_{s \rightarrow t, 1}), \quad o_t = \frac{c_t}{\|c_t\|_2},$$

and $\chi_{s,t} = 1$ when $\sin(\alpha_s - \alpha_t) \geq 0$, otherwise $\chi_{s,t} = -1$. The implemented channel entry and handoff points are

$$\begin{aligned} e_p &= c_s + \rho_{\text{source}} d_{s \rightarrow t} + \chi_{s,t} \delta_{\text{lane}} n_{s \rightarrow t}, \\ h_p &= c_t + \rho_{\text{hand}} o_t + \chi_{s,t} \delta_{\text{lane}} (-o_{t, 2}, o_{t, 1}), \end{aligned}$$

with channel direction $d_p = (h_p - e_p) / \|h_p - e_p\|_2$, channel normal $n_p = (-d_{p, 2}, d_{p, 1})$, and channel length $\ell_p = (h_p - e_p) \cdot d_p$.

The vector field is piecewise analytic. Core regions $\|x - c_b\|_2 \leq \rho_{\text{core}}$ use $A_b(x - c_b)$. Source annuli use $0.65A_b(x - c_b)$, and other non-channel background regions use $0.50A_b(x - c_b)$ for the nearest center c_b . Exit wedges for $p = (s, t)$ are source-neighborhood states outside the core with $\|x - c_s\|_2 \geq \rho_{\text{exit}}$ and $(x - c_s) \cdot d_{s \rightarrow t} \geq \|x - c_s\|_2 \cos \beta_{\text{exit}}$; they use

$$\dot{x} = v_{\text{exit}} d_{s \rightarrow t} - \lambda_{\text{exit}} ((x - c_s) \cdot n_{s \rightarrow t}) n_{s \rightarrow t}, \quad v_{\text{exit}} = 1.0, \quad \lambda_{\text{exit}} = 1.8;$$

and channel rectangles satisfying

$$0 \leq (x - e_p) \cdot d_p \leq \ell_p, \quad |(x - e_p) \cdot n_p| \leq w_{\text{chan}}$$

use

$$\dot{x} = v_{\text{chan}} d_p - \lambda_{\text{chan}} ((x - e_p) \cdot n_p) n_p, \quad v_{\text{chan}} = 1.55, \quad \lambda_{\text{chan}} = 2.8.$$

Other specialized systems. The arrested spiral system has a background spiral flow plus four Gaussian traps and an origin trap:

$$\dot{x} = -0.3x + 2.0Rx - 4.0 \sum_{i=1}^4 e^{-\|x - c_i\|_2^2 / (2 \cdot 0.25)} \frac{x - c_i}{0.25} - 1.5e^{-\|x\|_2^2 / 0.3} \frac{x}{0.15} - 0.02(x_1^3, x_2^3),$$

with $c_i \in \{(1.5, 0), (0, 1.8), (-1, -1), (0.8, -1.5)\}$. Its fifth basin is the origin trap, matching the benchmark manifest count.

The triple-well Duffing system uses $x = (q, p)$ and

$$V(q) = q^6/6 - q^4/2 + aq^2, \quad V'(q) = q^5 - 2q^3 + 2aq,$$

with $a = 0.3$, damping $\delta = 0.5$, rotation strength $\omega = 1.0$, and soft cubic confinement $-\epsilon u^3/s^3$ with $\epsilon = 0.003$ and $s = 4.0$:

$$\begin{aligned} \dot{q} &= (1 + \omega)p - \epsilon q^3/s^3, \\ \dot{p} &= -V'(q) - \delta p - \omega q - \epsilon p^3/s^3. \end{aligned} \tag{16}$$

The SNIC system is implemented in polar form and then converted to Cartesian coordinates. With $\varepsilon_{\text{num}} = 10^{-8}$, $r^2 = x_1^2 + x_2^2 + \varepsilon_{\text{num}}$, $r = \sqrt{r^2}$, $\theta = \text{atan2}(x_2, x_1)$, $c_\theta = x_1/(r + \varepsilon_{\text{num}})$, and $s_\theta = x_2/(r + \varepsilon_{\text{num}})$,

$$\dot{r} = r(1 - r^2) - 0.3 \cos(3\theta), \quad \dot{\theta} = 1 - 1.2 \cos(3\theta).$$

Table 7: The ten Dysts systems used in the $dt \times 30$ forecasting benchmark. The “stored step” column is 30 times the native Dysts integration interval. The final column gives the hand-specified diagnostic structure count used only to size the BD and SB transition parameterizations; it is a heuristic lobe, scroll, or equilibrium count, not basin truth.

Display name	Dimension	Native dt	Stored step $30dt$	Diagnostic count
Chua	3	$2.8474745791 \times 10^{-4}$	$8.5424237373 \times 10^{-3}$	2
Dadras	3	$6.5782963827 \times 10^{-4}$	$1.9734889148 \times 10^{-2}$	2
Dequan Li	3	$1.6763993999 \times 10^{-5}$	$5.0291981997 \times 10^{-4}$	3
Hadley	3	$2.9086847808 \times 10^{-4}$	$8.7260543424 \times 10^{-3}$	3
Lu–Chen–Cheng	3	$1.8469678280 \times 10^{-4}$	$5.5409034839 \times 10^{-3}$	4
Qi–Chen	3	$7.8371061844 \times 10^{-5}$	$2.3511318553 \times 10^{-3}$	2
Sakarya	3	$9.9704617436 \times 10^{-4}$	$2.9911385231 \times 10^{-2}$	2
San–Um–Srisuchinwong	3	$1.4933288881 \times 10^{-3}$	$4.4799866644 \times 10^{-2}$	2
Shimizu–Morioka	3	$2.4080013336 \times 10^{-3}$	$7.2240040007 \times 10^{-2}$	2
Wang–Sun	3	$5.3924987498 \times 10^{-3}$	$1.6177496249 \times 10^{-1}$	4

The Cartesian vector field is

$$\begin{aligned}\dot{x}_1 &= \dot{r}c_\theta - r\dot{\theta}s_\theta - 0.01x_1(x_1^2 + x_2^2) + 0.5x_2, \\ \dot{x}_2 &= \dot{r}s_\theta + r\dot{\theta}c_\theta - 0.01x_2(x_1^2 + x_2^2) - 0.5x_1.\end{aligned}$$

These explicit generators define the benchmark trajectories. Evaluation annotations use native labels for the two gated systems and nearest-estimated- center proxy labels after endpoint rollout for the catalog systems; the learned models observe only stored states.

C Dysts benchmark inventory and system definitions

This appendix lists the ten Dysts systems used in the paper-facing $dt \times 30$ forecasting benchmark. The systems are drawn from Dysts (Gilpin, 2021, 2023). We used the continuous-time right-hand sides and metadata from the pinned package version resolved in the project environment, `dysts` 0.96. All ten systems are three-dimensional autonomous flows. The equations below are written in unstandardized native Dysts coordinates (x, y, z) ; the training cache standardizes coordinates only after trajectories are generated.

Closed-form convention. For each system, the stored observations are generated from $\dot{x} = f_s(x)$ with observation interval $\Delta t = 30 dt_{\text{native}}$, where dt_{native} is the system-specific Dysts integration interval in table 7. The learner observes only the resulting stored states, not the vector field, continuous-time solver, or substeps. All ten systems have explicit closed-form right-hand sides in Dysts. Most are polynomial vector fields. The Chua and San–Um–Srisuchinwong systems include absolute-value terms, so they are piecewise smooth rather than globally analytic at the corresponding switching surfaces.

Chua. The Chua system uses the piecewise-linear diode nonlinearity

$$h(x) = m_1x + \frac{1}{2}(m_0 - m_1)(|x + 1| - |x - 1|),$$

with parameters

$$\alpha = 15.6, \quad \beta = 28, \quad m_0 = -1.142857, \quad m_1 = -0.71429.$$

The vector field is

$$\begin{aligned}\dot{x} &= \alpha(y - x - h(x)), \\ \dot{y} &= x - y + z, \\ \dot{z} &= -\beta y.\end{aligned}\tag{17}$$

Dadras. With parameters

$$c = 2, \quad e = 9, \quad o = 2.7, \quad p = 3, \quad r = 1.7,$$

the Dadras system is

$$\begin{aligned}\dot{x} &= y - px + oyz, \\ \dot{y} &= ry - xz + z, \\ \dot{z} &= cxy - ez.\end{aligned}\tag{18}$$

Dequan Li. With parameters

$$a = 40, \quad c = 1.833, \quad d = 0.16, \quad \varepsilon = 0.65, \quad f = 20, \quad k = 55,$$

the Dequan Li system is

$$\begin{aligned}\dot{x} &= a(y - x) + dxz, \\ \dot{y} &= kx + fy - xz, \\ \dot{z} &= cz + xy - \varepsilon x^2.\end{aligned}\tag{19}$$

Hadley. With parameters

$$a = 0.2, \quad b = 4, \quad f = 9, \quad g = 1,$$

the Hadley system is

$$\begin{aligned}\dot{x} &= -y^2 - z^2 - ax + af, \\ \dot{y} &= xy - bxz - y + g, \\ \dot{z} &= bxy + xz - z.\end{aligned}\tag{20}$$

Lu–Chen–Cheng. With parameters

$$a = -10, \quad b = -4, \quad c = 18.1,$$

the Lu–Chen–Cheng system is

$$\begin{aligned}\dot{x} &= -\frac{ab}{a+b}x - yz + c, \\ \dot{y} &= ay + xz, \\ \dot{z} &= bz + xy.\end{aligned}\tag{21}$$

Qi–Chen. With parameters

$$a = 38, \quad b = 2.666, \quad c = 80,$$

the Qi–Chen system is

$$\begin{aligned}\dot{x} &= a(y - x) + yz, \\ \dot{y} &= cx + y - xz, \\ \dot{z} &= xy - bz.\end{aligned}\tag{22}$$

Sakarya. With parameters

$$a = -1, \quad b = 1, \quad c = 1, \quad h = 1, \quad p = 1, \quad q = 0.4, \quad r = 0.3, \quad s = 1,$$

the Sakarya system is

$$\begin{aligned}\dot{x} &= ax + hy + syz, \\ \dot{y} &= -by - px + qxz, \\ \dot{z} &= cz - rxy.\end{aligned}\tag{23}$$

San–Um–Srisuchinwong. With parameter $a = 2$, the San–Um–Srisuchinwong system is

$$\begin{aligned}\dot{x} &= y - x, \\ \dot{y} &= -z \tanh(x), \\ \dot{z} &= -a + xy + |y|.\end{aligned}\tag{24}$$

Shimizu–Morioka. With parameters

$$a = 0.85, \quad b = 0.5,$$

the Shimizu–Morioka system is

$$\begin{aligned}\dot{x} &= y, \\ \dot{y} &= x - ay - xz, \\ \dot{z} &= -bz + x^2.\end{aligned}\tag{25}$$

Wang–Sun. With parameters

$$a = 0.2, \quad b = -0.01, \quad d = -0.4, \quad e = -1, \quad f = -1, \quad q = 1,$$

the Wang–Sun system is

$$\begin{aligned}\dot{x} &= ax + qyz, \\ \dot{y} &= bx + dy - xz, \\ \dot{z} &= ez + fxy.\end{aligned}\tag{26}$$

D Support definitions and family construction

All support objects are computed after or between training stages from $z = E_\theta(x)$. Basin labels, basin counts, and trajectory assignments do not define an exact support, merge masks, or route a rollout. The known count used to estimate catalog-system evaluation centers is therefore not an input to $F_{\text{abs}}^{\text{align}}$, although it is an input to the label and scoring-slice construction applied afterward. The paper uses two protocol-specific absolute-threshold families and does not assume that they are the same partition.

Absolute support. For every state,

$$m_{\text{abs},i}(x) = \mathbf{1}\{|z_i| > 10^{-3}\}, \quad S_{\text{abs}}(x) = \{i : m_{\text{abs},i}(x) = 1\}.$$

The 10^{-3} threshold is fixed in both experiments.

Deterministic greedy family merge. For a specified state collection and Jaccard threshold τ , distinct Boolean masks are counted and ordered from most to least frequent; equal frequencies use ascending byte order after `numpy.packbits` with its default big-bit ordering. During the greedy merge, each family is compared against the first mask that opened it. For a new mask m , compute

$$J(m, r_f) = \frac{|S(m) \cap S(r_f)|}{|S(m) \cup S(r_f)|}, \quad J(0, 0) = 1.$$

The mask joins the representative with largest J when that score is at least τ ; equal Jaccard scores choose the earliest-created family. Otherwise it starts a new family. These merge-time representatives are not averaged or updated, so family identifiers are meaningful only through their induced partition for that fitted collection. The alignment diagnostic retains these representatives. The staged route subsequently replaces each runtime representative with that family’s modal source-state support, as specified below.

Input: masks and counts from collection $\mathcal{K}$; threshold τ .

Initialize: ordered representative list $R \leftarrow []$.

For each distinct mask m : assign m to the best existing $r \in R$ if $J(m, r) \geq \tau$; otherwise append m to R .

Output: the fixed representatives and mask-to-family assignment.

Alignment family. $F_{\text{abs}}^{\text{align}}$ uses $\tau = 0.50$. At evaluation seed 42, it is fitted on all $128 \times 129 = 16,512$ states from 128 trajectories with 128 transitions each. The two gated environments supply native labels and centers. The 13 catalog environments supply neither, so the evaluator advances the 128 trajectory endpoints for 5,000 additional steps and estimates the known manifest number of centers by deterministic k -means. Initialization starts with the first converged endpoint and adds the endpoint farthest from its nearest current center; a farthest-point tie chooses the first endpoint index. Lloyd assignment ties choose the first center index, an empty cluster retains its prior center, and at most 25 updates follow; iteration stops early under PyTorch’s default `allclose` test. Every evaluation state is then assigned the nearest estimated center as a proxy label, again with first-center tie breaking.

Only after family fitting, each state’s center-margin proxy $d_2 - d_1$ is computed from its second-nearest and nearest native or estimated center. The evaluator uses NumPy’s default linear 75th-percentile rule within each observed evaluation label and retains all ties at the cutoff; labels with fewer than four states retain all their states. It does not equalize label counts or enforce an exact one-quarter subset. Most system-seed subsets contain 4,129–7,167 states. Triple-well Duffing retains all 16,512 because of tied margins, but all states also receive one proxy label, so its $H(B | F) = 0$ for every model is uninformative. Primary alignment aggregates use the 14 systems with at least two observed labels. Duffing’s raw rows are retained for an all-15-system sensitivity. The historical entropy reducer computes joint entropy minus family entropy with natural logarithms and $-p \log(p + 10^{-12})$ terms; values are reported in nats on this high-margin subset. Labels and centers do not enter the mask merge as features. The result is transductive because the fitting

collection contains the scored states and, for catalog systems, the endpoints used to estimate centers. In the main alignment table, the compact notation F_{abs} refers specifically to $F_{\text{abs}}^{\text{align}}$.

Low $H(B | F_{\text{abs}}^{\text{align}})$ means that the fitted partition leaves little uncertainty about the native or proxy label on that slice. The margin expresses separation under a nearest-center construction; it need not equal distance from a true basin boundary or separatrix. The result does not imply a frozen classifier for new states, one family per basin, or recovery near basin boundaries. The reported family count is the number of family IDs observed on the scored high-margin slice, not the total number formed over all evaluation states; it is compared descriptively with the number of represented evaluation labels.

Staged routing family. $F_{\text{abs}}^{\text{route}}$ uses $\tau = 0.40$ after stage one. The source artifact’s nominal 512-row route-fit packet is two exactly identical copies of 256 unique training-distribution trajectories, generated at seed offset 271,828. Each row contains 193 states and 192 transitions. The greedy family merge consumes all 193 supports, including the terminal state, for 98,816 nominal support observations. Only afterward are family counts, source centers, and exact mask-to-family keys restricted to the first 192 source states, for 98,304 nominal and 49,152 unique source transitions.

For runtime routing, each merged family’s representative is replaced by the modal support among that family’s source states; ties follow the first-seen source-key order. A family is retained for a local affine map when it has at least one source transition. During rollout, an exact source mask observed during fitting maps directly to its frozen family. An unseen mask is assigned to the modal source representative with greatest Jaccard overlap only if the overlap reaches 0.40; otherwise the rollout uses the frozen global K . The current latent’s support is recomputed and routed before every latent transition. A scheduled decode–re-encode event only refreshes that latent before the next transition; it is not the routing cadence. This modal-source replacement is part of the frozen source protocol; replacing it with the merge-opening mask or fitting 512 unique trajectories would define a new protocol.

The 0.40 routing merge and 0.50 alignment merge answer different questions and were applied to different state collections. Alignment statistics cannot be used to infer the route partition’s entropy, and the staged forecasting result cannot be read as an out-of-sample validation of the transductive alignment partition.

E Support-coordinate intervention table

Figure 1 plots the full curves for one Local-linear gates LISTA checkpoint at seed 0. Table 8 reports accumulated MSE at $H = 21$, defined here as the sum over rollout steps of the squared error averaged across state coordinates. This differs from the state-summed, through-horizon mean used by the headline forecasting table, so their numerical scales are not comparable. Candidate starts lie in the tie-inclusive per-native-label high center-margin subset and must retain their initial native label throughout the true 21-step future. The evaluator balances 100 starts across the three labels (34/33/33). Coordinate dropping and random support use exactly these same starts; random support uses 20 inactive-coordinate reassignments per start. Random-support summaries pool the resulting 2,000 dependent point–shuffle outcomes at each horizon. The margin is not a true basin-boundary distance. This one-system, one-model, one-seed experiment is descriptive and has no confirmatory significance test.

Table 8: Accumulated MSE (sum of per-coordinate step MSE) at $H = 21$ for descriptive support-coordinate interventions on the Local-linear gates LISTA checkpoint at seed 0. Drop rows summarize 100 starts; the random row pools 2,000 dependent point-shuffle outcomes. Lower is better; no confirmatory inference is attached.

Rollout	Mean $\pm$ SD	Median [IQR]
Standard	0.0158 $\pm$ 0.0127	0.0140 [0.0055, 0.0224]
Drop top-1	0.508 $\pm$ 0.647	0.218 [0.148, 0.677]
Drop top-2	1.37 $\pm$ 0.938	1.06 [0.802, 1.63]
Drop top-3	2.08 $\pm$ 1.10	1.80 [1.34, 2.10]
Drop top-5	3.26 $\pm$ 2.44	1.95 [1.61, 4.17]
Drop top-10	8.51 $\pm$ 6.83	5.16 [4.58, 8.11]
Random support	1.69 $\times 10^3 \pm 2.19 \times 10^3$	874 [377, 2.06 $\times 10^3$]

F Statistical testing protocol

Point estimates and significance annotations answer different questions. Forecasting and alignment entropy tables first summarize finite completed seeds within each system by `scipy.stats.trim_mean` with proportion 0.25, then take an arithmetic mean across systems. This sorts n values and discards $\lfloor n/4 \rfloor$ from each tail; complete 15-seed cells therefore average their central nine. We refer to this implementation as the interquartile mean (IQM). Forecasting averages all 15 controlled systems; primary alignment averages the 14 systems with at least two observed labels and retains an all-15 sensitivity. The descriptive family-count column instead averages per-system seed means. We do not pool trajectories, seeds, and systems into one inferential sample.

Forecasting metric and missingness. For start i , rollout step t , and state coordinate j , let

$$e_{i,t} = \sum_{j=1}^{d_x} (\hat{x}_{i,t,j} - x_{i,t,j})^2.$$

For the KAE rows used in formal comparisons, the reported MSE through horizon H takes the mean of finite $e_{i,t}$ over $t = 1, \dots, H$ within each start, then averages finite per-start means. It is therefore a state-summed squared error, not per-coordinate MSE. A rollout can contribute through its finite prefix even if it is nonfinite before H , so the KAE ranking does not enforce full-horizon finiteness. Local-linear mixture controls use the same finite-step convention; the unmatched classical DMD/EDMD evaluator requires a finite ordinary through- H mean for each contributing start, as detailed in [section A](#).

The error $E_{s,r,h}$ used below is also best-periodic: for every system s , seed r , model, and horizon h , the lowest MSE is selected on the same test trajectories over the fixed cadence grid. This test-set oracle is shared by the point estimates and formal KAE comparisons and should not be interpreted as validation-frozen deployment selection.

Controlled forecasting effects. For candidate model a , Dense MLP baseline, system s , seed r , and horizon h , the paired effect is

$$\Delta_{s,r,h}^a = \log_{10} E_{s,r,h}^a - \log_{10} E_{s,r,h}^{\text{Dense}}.$$

Only mutually finite positive MSEs are eligible. Within each system and candidate–horizon cell, a one-sided paired Wilcoxon signed-rank test uses the common seeds with alternative $\Delta < 0$. At least four finite pairs are required. Raw per-system p -values are Holm-corrected across eligible systems in that candidate–horizon family. A main-table star denotes corrected $p < 0.05$; every displayed controlled sparse-row forecasting cell passes on all 15/15 systems.

The per-system tables in [section G](#) display the arithmetic mean of the paired seed effects $\Delta_{s,r,h}^a$, together with the Wilcoxon/Holm p -value. The displayed effect is a mean, not the median used by an older forest-plot artifact. Because the signed-rank test depends on ranks whereas the mean does not, an extreme finite outlier can make their directions appear inconsistent; the Transfer-gated local-linear LISTA–BD H500 cell is the one such case, with 12/15 negative paired effects but a positive arithmetic mean.

Transductive alignment effects. The alignment partition $F_{\text{abs}}^{\text{align}}$ is fit on all states in an evaluation trajectory collection and scored on states at or above the within-label 75th center-margin percentile contained in that collection; ties are retained, so the subset need not contain exactly one quarter of a label, and no between-label count balancing is performed. Labels and centers are native for the two gated systems. For the 13 catalog systems they are nearest-center proxies obtained by deterministic clustering of long-rolled endpoints at the known benchmark count. The margin $d_2 - d_1$ is nearest-center separation, not a true boundary-distance measurement. $H(B | F_{\text{abs}}^{\text{align}})$ uses natural logarithms and is reported in nats. The seed is the paired unit within each system and the one-sided alternative is candidate $<$ Dense MLP. Holm correction is across eligible systems within each model–metric cell. Triple-well Duffing’s frozen proxy realizes one observed label, so its zero entropy for every model is uninformative and it is excluded from primary aggregates; the all-zero paired differences and undefined signed-rank statistic are consequences. Every displayed alignment star passes on all 14/14 eligible systems. The observed high-margin-slice family count has no directional test because closeness to the number of represented evaluation labels, rather than monotone increase, is the relevant descriptive comparison. These tests quantify model differences in the transductive diagnostic; they do not turn it into an out-of-sample classification experiment.

Dysts forecasting effects. For each Dysts system and horizon, seeds are first summarized within system:

$$I_{s,h}^a = \text{IQM}_r\{E_{s,r,h}^a\}, \quad I_{s,h}^{\text{Dense}} = \text{IQM}_r\{E_{s,r,h}^{\text{Dense}}\}.$$

Each system contributes one paired effect

$$d_{s,h} = \log_{10} I_{s,h}^a - \log_{10} I_{s,h}^{\text{Dense}}.$$

The compact-table superscript uses an exact one-sided sign test on the number of retained systems with $d_{s,h} < 0$. Holm correction is one family of 40 tests: five non-Dense models at each of the eight evaluated horizons 100, 500, 1000, 1500, 2000, 3000, 4000, 5000, not only the three displayed columns. With 10 retained systems and this correction, a displayed star corresponds to improvement on all 10/10 systems in that cell. LISTA is starred at H100 and H4000; LISTA–BD at H2000 and H4000; Sparse MLP–BD only at H4000; Sparse MLP at H100 and H2000; and LISTA–SB at none of the three displayed horizons.

Descriptive comparisons. The standalone DMD, EDMD, and local-linear controls configure three seeds and use independently generated evaluation trajectories; their per-system summaries omit nonfinite seed values, so they are not included in paired KAE tests. The staged support-family result reports 225 system–seed pairs nested within 15 systems, plus MSE ratios, without a

confirmatory p -value. Its checkpoint score applies the same finite-prefix state-summed error reducer as the reported table, minimizes separately over the eight cadences at H100, H500, and H1000, and averages those three minima. The 32 selector starts are the first 32 of the 100 reported starts, whereas the global baseline selects its checkpoint on 16 distinct starts with an H200 final-error proxy. This 32% staged overlap and selector asymmetry make the comparison optimistic and preclude interpreting the paired win counts as held-out causal evidence. The Local-linear gates coordinate intervention uses one LISTA checkpoint at seed 0; bootstrap intervals summarize its 100 starts, while random-support medians and interquartile ranges pool 2,000 dependent point-shuffle outcomes per horizon. These describe within-checkpoint variability only.

Holm correction and assumptions. For each formal family, eligible raw p -values are sorted, multiplied by the number of remaining hypotheses, made monotone by a running maximum, and clipped at one. Paired Wilcoxon tests assume the common seed differences are the relevant exchangeable units within a controlled system. Dysts sign tests treat benchmark systems as the independent units. All conclusions condition on the fixed benchmark roster. Evaluation-set period selection, finite-prefix omission, staged checkpoint-selection overlap, selector asymmetry, and transductive family fitting are protocol limitations independent of the multiple-comparison correction.

G Full per-system controlled-benchmark tables

Tables 9–11 report the per-system forecasting effects and Holm-corrected Wilcoxon p -values underlying the controlled columns of table 1. Each displayed Δ is the arithmetic mean over common completed seeds of the paired $\log_{10}$ -MSE difference, candidate minus Dense MLP. The associated test is the one-sided paired Wilcoxon test on those seed effects. Bold entries pass Holm-corrected $p < 0.05$ in favor of the candidate. The rank test and arithmetic mean need not have the same sign under an extreme outlier: at H500 for Transfer-gated local-linear LISTA-BD, 12/15 paired seed effects are negative, but one divergent finite candidate seed makes the displayed mean positive.

H Dysts forecasting diagnostics

The collected experiment artifacts contain per-(system, seed) Dysts MSE rows. Table 1 reports arithmetic means across systems after first applying `scipy.stats.trim_mean` with proportion 0.25 to the 15 seeds within each system, i.e., averaging the central nine after discarding three values from each tail. We call this seed summary IQM. The soft-block LISTA row is an architecture-specific ablation: it uses two LISTA loops and a sign-split output rather than the one-loop/ReLU recipe of LISTA and LISTA-BD, though all rows use the same 100,000-step budget. Its source packet contained 12 systems and the paper uses the fixed retained 10-system subset. LISTA-SB improves on Dense MLP in aggregate but has no corrected Dysts significance marker.

As a robustness check on the cross-system aggregation, table 13 and figure 9 replace the final arithmetic mean across systems with the same 0.25-per-tail trimmed mean across the 10 retained systems, averaging the central six after dropping two systems from each tail. This IQM-over-IQM view asks for the typical retained-system performance and trims both unusually easy and unusually hard chaotic-flow systems. With the repaired block-diagonal Sparse MLP run, Sparse MLP-BD has the lowest MSE at every displayed horizon from H100 through H5000. This should be read as an aggregation-sensitivity result for the external forecasting benchmark, not as a replacement for the average-case Dysts table in the main text.

Table 9: Per-system forecasting effects at $H = 100$ on the 15-system controlled benchmark. Δ is the arithmetic mean paired $\log_{10}$ -MSE difference over common seeds (candidate minus Dense MLP), not a median; p_H is the one-sided paired Wilcoxon p -value Holm-corrected across retained systems within each candidate column.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
Local-linear gates	-2.16	$<10^{-3}$	-2.19	$<10^{-3}$	-2.20	$<10^{-3}$	-2.33	$<10^{-3}$	-2.52	$<10^{-3}$
Transfer-gated local-linear	-0.27	0.001	-0.21	0.008	-0.31	0.007	-0.37	$<10^{-3}$	-0.27	0.004
Arrested spiral	-1.85	$<10^{-3}$	-2.09	$<10^{-3}$	-2.03	$<10^{-3}$	-1.43	$<10^{-3}$	-1.90	$<10^{-3}$
Asymmetric three-well	-2.00	$<10^{-3}$	-1.92	$<10^{-3}$	-1.75	$<10^{-3}$	-2.43	$<10^{-3}$	-2.44	$<10^{-3}$
High-cross three-well	-1.05	$<10^{-3}$	-1.12	$<10^{-3}$	-1.19	$<10^{-3}$	-0.95	$<10^{-3}$	-1.17	$<10^{-3}$
Hexagonal six-well	-1.76	$<10^{-3}$	-1.73	$<10^{-3}$	-1.85	$<10^{-3}$	-1.70	$<10^{-3}$	-1.83	$<10^{-3}$
Octagonal eight-well	-1.85	$<10^{-3}$	-1.90	$<10^{-3}$	-1.80	$<10^{-3}$	-1.66	$<10^{-3}$	-1.63	$<10^{-3}$
Pentagonal five-well	-1.86	$<10^{-3}$	-1.58	$<10^{-3}$	-1.20	$<10^{-3}$	-1.53	$<10^{-3}$	-2.16	$<10^{-3}$
Square four-well	-1.44	$<10^{-3}$	-1.62	$<10^{-3}$	-1.58	$<10^{-3}$	-1.26	$<10^{-3}$	-1.02	$<10^{-3}$
Triple-well Duffing	-0.54	$<10^{-3}$	-0.51	0.008	-0.35	0.007	-0.90	$<10^{-3}$	-0.78	$<10^{-3}$
SNIC multi-attractor	-2.01	$<10^{-3}$	-2.03	$<10^{-3}$	-1.91	$<10^{-3}$	-2.21	$<10^{-3}$	-2.13	$<10^{-3}$
Transition-routes four-well	-2.21	$<10^{-3}$	-1.77	$<10^{-3}$	-1.90	$<10^{-3}$	-2.14	$<10^{-3}$	-2.47	$<10^{-3}$
Depth-gradient four-well	-1.39	$<10^{-3}$	-1.33	$<10^{-3}$	-1.37	$<10^{-3}$	-0.94	$<10^{-3}$	-0.82	$<10^{-3}$
Diamond four-well	-2.00	$<10^{-3}$	-1.80	$<10^{-3}$	-1.84	$<10^{-3}$	-1.51	$<10^{-3}$	-1.67	$<10^{-3}$
L-shaped five-well	-2.23	$<10^{-3}$	-1.94	$<10^{-3}$	-1.90	$<10^{-3}$	-1.85	$<10^{-3}$	-2.23	$<10^{-3}$
<i>Significant/eligible</i>	$K=15 / N=15$		$K=15 / N=15$		$K=15 / N=15$		$K=15 / N=15$		$K=15 / N=15$	

Table 10: Per-system forecasting effects at $H = 500$. Effect and test definitions match [table 9](#).

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
Local-linear gates	-2.01	$<10^{-3}$	-2.24	$<10^{-3}$	-2.27	$<10^{-3}$	-2.29	$<10^{-3}$	-2.52	$<10^{-3}$
Transfer-gated local-linear	-0.08	0.013	+1.19	0.008	-0.21	$<10^{-3}$	-0.17	$<10^{-3}$	-0.18	$<10^{-3}$
Arrested spiral	-2.26	$<10^{-3}$	-2.57	$<10^{-3}$	-2.57	$<10^{-3}$	-1.72	$<10^{-3}$	-2.33	$<10^{-3}$
Asymmetric three-well	-2.27	$<10^{-3}$	-2.12	$<10^{-3}$	-1.94	$<10^{-3}$	-3.11	$<10^{-3}$	-3.15	$<10^{-3}$
High-cross three-well	-0.93	$<10^{-3}$	-0.97	$<10^{-3}$	-1.03	$<10^{-3}$	-0.86	$<10^{-3}$	-1.26	$<10^{-3}$
Hexagonal six-well	-2.01	$<10^{-3}$	-2.01	$<10^{-3}$	-2.18	$<10^{-3}$	-1.95	$<10^{-3}$	-2.16	$<10^{-3}$
Octagonal eight-well	-1.94	$<10^{-3}$	-2.09	$<10^{-3}$	-1.91	$<10^{-3}$	-1.78	$<10^{-3}$	-1.74	$<10^{-3}$
Pentagonal five-well	-2.48	$<10^{-3}$	-2.10	$<10^{-3}$	-1.61	$<10^{-3}$	-2.02	$<10^{-3}$	-2.77	$<10^{-3}$
Square four-well	-1.60	$<10^{-3}$	-1.89	$<10^{-3}$	-1.81	$<10^{-3}$	-1.55	$<10^{-3}$	-1.17	$<10^{-3}$
Triple-well Duffing	-0.83	$<10^{-3}$	-0.81	$<10^{-3}$	-0.65	$<10^{-3}$	-1.10	$<10^{-3}$	-1.05	$<10^{-3}$
SNIC multi-attractor	-2.30	$<10^{-3}$	-2.45	$<10^{-3}$	-2.31	$<10^{-3}$	-2.56	$<10^{-3}$	-2.49	$<10^{-3}$
Transition-routes four-well	-1.73	$<10^{-3}$	-1.52	$<10^{-3}$	-1.49	$<10^{-3}$	-1.55	$<10^{-3}$	-1.86	$<10^{-3}$
Depth-gradient four-well	-1.63	$<10^{-3}$	-1.45	$<10^{-3}$	-1.47	$<10^{-3}$	-1.17	$<10^{-3}$	-0.92	$<10^{-3}$
Diamond four-well	-1.77	$<10^{-3}$	-1.58	$<10^{-3}$	-1.40	$<10^{-3}$	-1.28	$<10^{-3}$	-1.75	$<10^{-3}$
L-shaped five-well	-2.98	$<10^{-3}$	-2.04	$<10^{-3}$	-2.03	$<10^{-3}$	-2.27	$<10^{-3}$	-2.96	$<10^{-3}$
<i>Significant/eligible</i>	$K=15 / N=15$		$K=15 / N=15$		$K=15 / N=15$		$K=15 / N=15$		$K=15 / N=15$	

Table 11: Per-system forecasting effects at $H = 1000$. Effect and test definitions match [table 9](#).

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
Local-linear gates	-1.98	$< 10^{-3}$	-2.24	$< 10^{-3}$	-2.29	$< 10^{-3}$	-2.28	$< 10^{-3}$	-2.52	$< 10^{-3}$
Transfer-gated local-linear	-0.08	0.013	-0.15	$< 10^{-3}$	-0.20	$< 10^{-3}$	-0.16	$< 10^{-3}$	-0.17	$< 10^{-3}$
Arrested spiral	-2.32	$< 10^{-3}$	-2.65	$< 10^{-3}$	-2.66	$< 10^{-3}$	-1.76	$< 10^{-3}$	-2.39	$< 10^{-3}$
Asymmetric three-well	-2.27	$< 10^{-3}$	-2.12	$< 10^{-3}$	-1.93	$< 10^{-3}$	-3.16	$< 10^{-3}$	-3.22	$< 10^{-3}$
High-cross three-well	-0.92	$< 10^{-3}$	-0.96	$< 10^{-3}$	-0.99	$< 10^{-3}$	-0.85	$< 10^{-3}$	-1.26	$< 10^{-3}$
Hexagonal six-well	-2.02	$< 10^{-3}$	-2.02	$< 10^{-3}$	-2.19	$< 10^{-3}$	-1.96	$< 10^{-3}$	-2.18	$< 10^{-3}$
Octagonal eight-well	-1.89	$< 10^{-3}$	-2.05	$< 10^{-3}$	-1.86	$< 10^{-3}$	-1.73	$< 10^{-3}$	-1.70	$< 10^{-3}$
Pentagonal five-well	-2.55	$< 10^{-3}$	-2.15	$< 10^{-3}$	-1.64	$< 10^{-3}$	-2.06	$< 10^{-3}$	-2.84	$< 10^{-3}$
Square four-well	-1.62	$< 10^{-3}$	-1.92	$< 10^{-3}$	-1.84	$< 10^{-3}$	-1.60	$< 10^{-3}$	-1.21	$< 10^{-3}$
Triple-well Duffing	-0.80	$< 10^{-3}$	-0.73	$< 10^{-3}$	-0.64	$< 10^{-3}$	-1.05	$< 10^{-3}$	-1.02	$< 10^{-3}$
SNIC multi-attractor	-2.31	$< 10^{-3}$	-2.49	$< 10^{-3}$	-2.35	$< 10^{-3}$	-2.59	$< 10^{-3}$	-2.53	$< 10^{-3}$
Transition-routes four-well	-1.54	$< 10^{-3}$	-1.38	$< 10^{-3}$	-1.36	$< 10^{-3}$	-1.40	$< 10^{-3}$	-1.62	$< 10^{-3}$
Depth-gradient four-well	-1.67	$< 10^{-3}$	-1.48	$< 10^{-3}$	-1.50	$< 10^{-3}$	-1.20	$< 10^{-3}$	-0.94	$< 10^{-3}$
Diamond four-well	-1.76	$< 10^{-3}$	-1.53	$< 10^{-3}$	-1.33	$< 10^{-3}$	-1.20	$< 10^{-3}$	-1.74	$< 10^{-3}$
L-shaped five-well	-3.08	$< 10^{-3}$	-2.03	$< 10^{-3}$	-2.07	$< 10^{-3}$	-2.26	$< 10^{-3}$	-3.05	$< 10^{-3}$
<i>Significant/eligible</i>	$K=15 / N=15$		$K=15 / N=15$		$K=15 / N=15$		$K=15 / N=15$		$K=15 / N=15$	

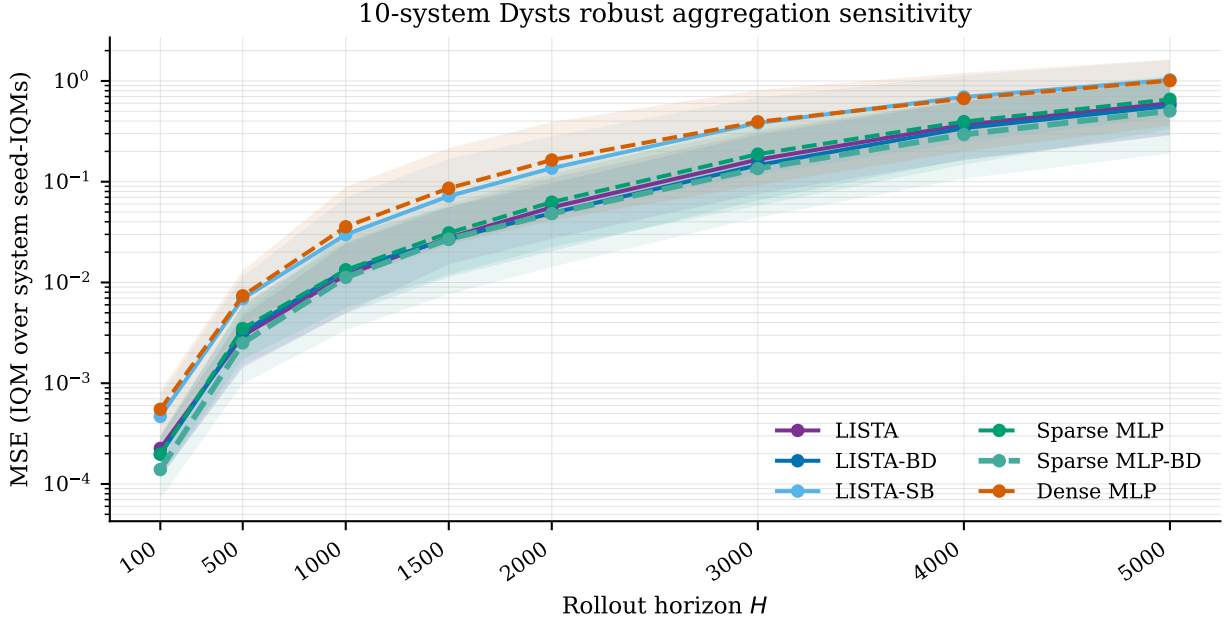


Figure 9: Dysts $dt \times 30$ IQM-over-IQM horizon sensitivity. Lines show IQM across retained systems after first summarizing seeds within each system by IQM. Shaded bands show the inter-system IQR of the per-system seed-IQM MSE values, not a confidence interval. With the repaired block-diagonal Sparse MLP run, Sparse MLP-BD is the best robust aggregate at every displayed horizon.

Table 12: Per-system transductive $H(B | F_{\text{abs}}^{\text{align}})$ effects, in nats, on the within-label high center-margin slice. Families are fitted on all states in each evaluation trajectory collection and scored at or above each label’s 75th margin percentile, retaining ties; native or nearest-estimated-center proxy labels select only that subset, without equalizing label counts. The known benchmark count is used to estimate catalog-system centers but does not enter family formation. The family-count companion is the number of family IDs observed on the scored subset. The one-sided paired Wilcoxon alternative is candidate entropy < Dense MLP entropy, with Holm correction across eligible systems. Triple-well Duffing’s proxy realizes one label and is excluded from primary alignment aggregation; its zero paired differences are a consequence. Displayed alignment stars pass on all 14/14 eligible systems.

System	LISTA		LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
Local-linear gates	-1.09	<10 ⁻³	-1.09	<10 ⁻³	-1.09	<10 ⁻³	-1.09	<10 ⁻³	-1.09	<10 ⁻³
Transfer-gated local-linear	-1.10	<10 ⁻³	-1.10	<10 ⁻³	-1.10	<10 ⁻³	-1.10	<10 ⁻³	-1.10	<10 ⁻³
Arrested spiral	-1.04	0.001	-0.92	0.001	-0.98	0.001	-0.27	0.026	-0.98	0.001
Asymmetric three-well	-1.09	<10 ⁻³	-1.09	<10 ⁻³	-1.09	<10 ⁻³	-1.09	<10 ⁻³	-1.09	<10 ⁻³
High-cross three-well	-0.95	<10 ⁻³	-0.79	0.001	-1.01	<10 ⁻³	-0.20	0.026	-0.70	0.001
Hexagonal six-well	-1.40	0.001	-1.31	0.001	-1.17	0.001	-0.77	0.001	-0.82	0.001
Octagonal eight-well	-1.34	0.001	-1.25	0.001	-1.34	0.001	-1.08	0.001	-1.09	0.001
Pentagonal five-well	-1.57	<10 ⁻³	-1.51	<10 ⁻³	-1.41	0.001	-0.95	0.001	-1.34	0.001
Square four-well	-1.34	<10 ⁻³	-1.36	<10 ⁻³	-1.34	<10 ⁻³	-1.32	<10 ⁻³	-1.36	<10 ⁻³
Triple-well Duffing	+0.00	–	+0.00	–	+0.00	–	+0.00	–	+0.00	–
SNIC multi-attractor	-1.09	<10 ⁻³	-1.09	<10 ⁻³	-1.09	<10 ⁻³	-1.09	<10 ⁻³	-1.09	<10 ⁻³
Transition-routes four-well	-1.39	<10 ⁻³	-1.39	<10 ⁻³	-1.39	<10 ⁻³	-1.39	<10 ⁻³	-1.39	<10 ⁻³
Depth-gradient four-well	-1.30	<10 ⁻³	-1.34	<10 ⁻³	-1.36	<10 ⁻³	-1.24	0.001	-1.36	<10 ⁻³
Diamond four-well	-1.33	<10 ⁻³	-1.33	<10 ⁻³	-1.36	<10 ⁻³	-1.36	<10 ⁻³	-1.36	<10 ⁻³
L-shaped five-well	-1.01	0.001	-0.96	0.001	-0.99	0.001	-0.92	<10 ⁻³	-0.92	<10 ⁻³
<i>Significant/eligible</i>	<i>K=14 / N=14</i>		<i>K=14 / N=14</i>		<i>K=14 / N=14</i>		<i>K=14 / N=14</i>		<i>K=14 / N=14</i>	

Table 13: Robust Dysts $dt \times 30$ forecasting aggregation. Each system first contributes the central-nine mean of 15 seeds; entries then average the central six of 10 retained systems. Lower is better; **bold** marks the best row in each horizon column.

Model	H100	H500	H1000	H1500	H2000	H3000	H4000	H5000
LISTA	2.26×10^{-4}	0.00296	0.0119	0.0278	0.0557	0.166	0.367	0.602
LISTA-BD	1.98×10^{-4}	0.00322	0.0133	0.0269	0.0488	0.146	0.340	0.564
LISTA-SB	4.68×10^{-4}	0.00691	0.0299	0.0719	0.136	0.383	0.694	1.02
Sparse MLP	1.98×10^{-4}	0.00349	0.0133	0.0310	0.0627	0.188	0.395	0.658
Sparse MLP-BD	1.39×10^{-4}	0.00251	0.0112	0.0270	0.0482	0.135	0.294	0.504
Dense MLP	5.51×10^{-4}	0.00737	0.0357	0.0861	0.164	0.393	0.669	1.01

Table 14: Scale-normalized Dyts forecasting summary. Entries are arithmetic means across systems of the candidate per-system seed-IQM MSE divided by the Dense MLP per-system seed-IQM MSE; lower than 1 favors the candidate. This table gives the average-case ratio view corresponding to [table 1](#). LISTA-SB is retained as an architecture-specific two-loop/sign-split diagnostic row.

Model	H100	H500	H1000	H1500	H2000	H3000	H4000	H5000
LISTA	0.491	0.426	0.440	0.475	0.493	0.533	0.574	0.596
LISTA-BD	0.527	0.485	0.475	0.469	0.465	0.499	0.560	0.593
LISTA-SB	1.065	1.011	1.080	1.196	1.300	1.395	1.137	1.031
Sparse MLP	0.427	0.459	0.486	0.552	0.570	0.604	0.660	0.710
Sparse MLP-BD	0.614	0.496	0.458	0.422	0.423	0.433	0.477	0.513
Dense MLP	1.000	1.000	1.000	1.000	1.000	1.000	1.000	1.000

I Additional background

I.1 Koopman operators and finite-dimensional approximations

Consider the stored-step map used in [section 2](#),

$$x_{k+1} = F_{\Delta t}(x_k), \quad x_k \in \mathcal{X} \subseteq \mathbb{R}^{d_x}. \quad (27)$$

Koopman operator theory studies the evolution of observables rather than the evolution of the state directly. An observable is a measurement function $g : \mathcal{X} \rightarrow \mathbb{R}$, and the Koopman operator $\mathcal{K}$ acts by composition:

$$(\mathcal{K}g)(x) = g(F_{\Delta t}(x)). \quad (28)$$

The map $F_{\Delta t}$ may be nonlinear, but $\mathcal{K}$ is linear on the function space of observables. If a finite vector of observables

$$\Phi(x) = [g_1(x), \dots, g_{d_z}(x)]^\top \quad (29)$$

spans a Koopman-invariant subspace, then there is a matrix $K \in \mathbb{R}^{d_z \times d_z}$ such that

$$\Phi(x_{k+1}) = \Phi(F_{\Delta t}(x_k)) = \mathcal{K}\Phi(x_k) = K\Phi(x_k). \quad (30)$$

In that case the nonlinear dynamics are exactly linear in the observable coordinates $z_k = \Phi(x_k)$. Learning Koopman models can therefore be viewed as searching for observables whose span is invariant enough to support useful finite-dimensional linear prediction.

In practice, the infinite-dimensional operator is approximated from data. Given snapshot pairs $\{(x_k, x_{k+1})\}$ and a chosen feature map Φ , dynamic mode decomposition and extended dynamic mode decomposition (eDMD) estimate a finite transition by least squares ([Schmid, 2010](#); [Rowley et al., 2009](#); [Tu et al., 2014](#); [Williams et al., 2015, 2016](#)):

$$K = \arg \min_{\tilde{K} \in \mathbb{R}^{d_z \times d_z}} \sum_k \left\| \Phi(x_{k+1}) - \tilde{K}\Phi(x_k) \right\|_2^2. \quad (31)$$

DMD is the special case $\Phi(x) = x$, so it fits a linear state-space surrogate without a learned nonlinear lift. eDMD extends this by choosing a richer observable dictionary. Koopman autoencoders replace the fixed observable dictionary with a learned encoder and decoder, which is the setting used in this paper.

I.2 Why multibasin systems may benefit from local structure

Near appropriate attracting sets, classical linearization and Koopman eigenfunction constructions can produce useful local or basin-restricted coordinates. Global results are more restrictive: multi-attractor systems should not in general be assumed to admit one exact finite-dimensional linearization with a continuous, state-reconstructing encoder–decoder pair (Lan and Mezić, 2013; Brunton et al., 2016; Pan and Duraisamy, 2024; Liu et al., 2025). These results motivate local structure, but they do not prove that every learned global KAE must fail or that sparse supports recover the theoretical local coordinates.

For the present paper, a learned finite-dimensional KAE is treated as an approximation. Sparse supports provide one possible discrete and inspectable key: a support may index a basin part or local predictive chart while coefficient values carry within-chart information. Whether that interpretation holds is an empirical question tested by the transductive alignment and staged forecasting experiments.

I.3 Koopman autoencoders and periodic re-encoding

A standard Koopman autoencoder learns an encoder $E_\theta : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_z}$, decoder $D_\phi : \mathbb{R}^{d_z} \rightarrow \mathbb{R}^{d_x}$, and latent transition K so that

$$z_k = E_\theta(x_k), \quad z_{k+1} \approx Kz_k, \quad x_k \approx D_\phi(z_k). \quad (32)$$

A schematic one-step objective has the form

$$\min_{E_\theta, D_\phi, K} \sum_k \|x_k - D_\phi(E_\theta(x_k))\|_2^2 + \lambda_{\text{lin}} \|E_\theta(x_{k+1}) - KE_\theta(x_k)\|_2^2. \quad (33)$$

This objective is useful as a reference point, but it is not sufficient for the present study. It does not directly train decoded multi-step rollouts, does not impose a sparse support object, and does not test whether the support itself selects a useful local predictive rule. The training objective in [section 3.3](#) therefore keeps the autoencoding and latent-linearity terms but adds decoded rollout prediction over windows, sparsity on the rolled latent states, and optional transition-structure penalties.

Periodic re-encoding interrupts a latent rollout after a fixed number of stored observation steps. For period m ,

$$\tilde{z}_{k+m} = K^m z_k, \quad \tilde{x}_{k+m} = D_\phi(\tilde{z}_{k+m}), \quad z_{k+m} = E_\theta(\tilde{x}_{k+m}). \quad (34)$$

This is the same mechanism defined in [equation \(12\)](#). It uses only the model’s own predicted state and is therefore not a teacher-forced correction. The point is to refresh the latent embedding and, in sparse models, to refresh the support as the predicted state moves through regions where a different local linear description may be more appropriate (Fathi et al., 2024).

I.4 Sparse coding and LISTA

Classical sparse coding seeks a code z that reconstructs a signal from a small set of active dictionary atoms:

$$z^\star(x) = \arg \min_{z \in \mathbb{R}^{d_z}} \frac{1}{2} \|x - Dz\|_2^2 + \lambda_{\text{sc}} \|z\|_1, \quad \lambda_{\text{sc}} > 0, \quad (35)$$

where D is a dictionary and λ_{sc} controls sparsity (Olshausen and Field, 1996; Chen et al., 1998; Donoho and Elad, 2003). For a fixed dictionary, and under the usual uniqueness and non-boundary

conditions for the Lasso solution, the active set and signs are locally constant over regions of the input space, while the coefficient values vary continuously within a fixed active set. The decoder reconstruction then lies in the span of the selected dictionary columns, giving a union-of-subspaces geometry. Each exact support can index a local chart, and the greedy Jaccard procedure in [section 3.4](#) groups nearby Boolean support vectors into support families when threshold noise or boundary effects fragment the exact masks.

LISTA replaces an iterative sparse-coding solver with a learned finite-depth shrinkage network ([Gregor and LeCun, 2010](#)). In the notation of [equation \(6\)](#), the encoder forms a pre-code from the input and repeatedly applies learned affine refinement followed by soft thresholding. In this paper, the encoder is trained jointly with the Koopman rollout objective rather than solving the sparse-coding problem at evaluation time. Thresholding turns the latent into continuous coefficients plus an explicit support. The experiments ask whether absolute-threshold support families align transductively with evaluation-only native or nearest-center-proxy labels and whether a separately fitted training-distribution family partition can route useful local affine predictors.

J Preliminary coauthor checklist

Status. This is a factual pre-submission record for coauthor review, not the official venue questionnaire. Application of the final venue style determines the official question set, numbering, and response format; these answers must then be reconciled with that form. Nothing here asserts that a public artifact, anonymous URL, DOI, or release has been completed.

Claims and evidence. The manuscript makes three bounded empirical claims: sparse KAE rows improve forecasting over the reported Dense MLP KAE rows; LISTA support families align transductively with evaluation-only native or nearest-center-proxy labels on a controlled high center-margin slice; and a staged training-distribution support-family route has descriptively lower controlled-forecasting error than a same-budget global- K LISTA baseline under an optimistic asymmetric selector. The paper states no new theorem or proof, no exact global linearization, and no out-of-sample basin-discovery guarantee. Benchmark definitions and training/evaluation protocols are in [sections A to C](#); support construction and inference are in [sections D and F](#).

Limits on interpretation.

- Best-periodic cadence is chosen on the test trajectories, and the MSE reducer omits nonfinite steps, so finite prefixes of divergent rollouts can contribute.
- Alignment fits families on all states in each evaluation trajectory collection. Two gated systems use native labels and centers; the other 13 use deterministic endpoint-estimated centers at the known benchmark count and nearest-center proxy labels. Scoring retains states at or above each label’s 75th center-margin percentile, including ties, so it need not retain exactly one quarter; Triple-well Duffing retains all 16,512 states. This margin need not equal true basin-boundary distance. Labels and centers select only the scored subset, label sample counts are not equalized, and the reported family count includes only family IDs observed on that subset. Conditional entropy uses natural logarithms and is reported in nats. The result is transductive and two-dimensional; alignment uses Jaccard threshold 0.50, while the separate staged route uses 0.40.
- The staged route-fit packet has 512 configured rows but only 256 unique trajectories because one batch is duplicated exactly. Its checkpoint selector uses the first 32/100 reported starts, while the global baseline uses a distinct 16-start, H200 final-error selector. Thus the 225 nested system–seed comparisons are optimistic and descriptive, not a clean held-out or causal comparison.
- Standalone controls use three seeds and unmatched trajectories; the intervention covers one checkpoint.
- LISTA decoders are atom-normalized whereas MLP decoders are not. Dysts LISTA–SB is a same-budget two-loop/sign-split ablation, unlike the one-loop/ReLU LISTA and LISTA–BD rows. Structured-transition rows use a known controlled-benchmark basin count or a hand-set Dysts lobe/scroll/equilibrium count only to set the number of blocks.

Reproducibility and access status. The working repository contains explicit controlled-system vector fields, benchmark adapters, training and evaluation code, seeds, locked dependencies, tests, manuscript sources, and paper-facing aggregate artifacts. Controlled trajectories are regenerable from the explicit analytic or piecewise-analytic systems; Dysts trajectories are regenerable from the documented package and cache protocol. Exact model, metric, split, cadence, support, and standalone-control recipes are recorded in [sections A and D](#). Bitwise equality across GPU models

and numerical-library versions is not claimed. No verified public anonymous archive or DOI is identified in the manuscript, so open code/data availability is currently answered *No*.

Compute accounting. The following are conservative SLURM allocation ceilings, not measured consumption or energy estimates:

- Controlled training comprises $15 \times 6 \times 15 = 1350$ runs, each requesting one cluster-assigned GPU, four CPU cores, 16 GB host memory, and at most 24 hours: at most 32,400 GPU-hours and 129,600 allocated CPU-core-hours.
- Dysts training comprises $10 \times 6 \times 15 = 900$ runs packed up to 12 per allocation, with one GPU, four CPU cores, 16 GB, and at most 72 hours per full pack: at most 5,400 GPU-hours and 21,600 allocated CPU-core-hours.
- The staged routing packet comprises $15 \times 15 = 225$ runs, each requesting one GPU, four CPU cores, 24 GB, and at most three hours: at most 675 GPU-hours and 2,700 allocated CPU-core-hours.
- Retained Dysts cache/evaluation jobs add approximately 4,700 CPU-core-hours; controlled support diagnostics add approximately 290 CPU-core-hours; the one-checkpoint intervention allocation adds at most 3 GPU-hours.

The GPU model was scheduler-assigned and is therefore unspecified. These ceilings exclude superseded historical packets and can substantially exceed actual use because jobs may finish early and packed runs share allocations.

Assets, attribution, and licenses. The controlled benchmark contains 15 author-retained analytic or piecewise-analytic systems. Claude Opus 4.5 assisted system ideation; human authors audited the retained definitions and metadata before their experimental use. The LLM is not part of model training, rollout, routing, metric computation, or statistical testing. LLM assistance also contributed to manuscript and checklist editing; the authors remain responsible for the equations, protocols, and numerical claims. The external benchmark uses right-hand sides and metadata from *dysts* 0.96, with project-generated standardized trajectories, and the manuscript cites the associated Gilpin papers. Local package metadata is not sufficient to state a definitive redistribution license: it contains an MIT license classifier but a bundled Apache-2.0 license text. The authoritative Dysts license and any system-level attribution requirements therefore remain unresolved. The repository itself has a top-level MIT license, but no public artifact or separate license for generated benchmark definitions, figures, tables, caches, or checkpoints is claimed here.

Ethics, human subjects, and broader impact. The experiments use simulated dynamical-system states and public benchmark definitions, with no participants, crowdworkers, personal data, private records, or human-generated labels. Human-subject review, consent, compensation, and IRB approval are therefore not applicable. This is methodological forecasting research, not a validated safety-critical control system. Learned support families can be misread as physical or causal regimes, and failures near boundaries or in untested high-dimensional systems could be consequential; downstream use requires independent validation and safeguards.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron C Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in Neural Information Processing Systems*, 34, 2021.

- Omri Azencot, N. Benjamin Erichson, Vanessa Lin, and Michael Mahoney. Forecasting Sequential Data Using Consistent Koopman Autoencoders. In *Proceedings of the 37th International Conference on Machine Learning*, pages 475–485. PMLR, November 2020.
- Amir Beck and Marc Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM Journal on Imaging Sciences*, 2(1):183–202, 2009. doi: 10.1137/080716542.
- Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control. *PLOS ONE*, 11(2), February 2016. ISSN 1932-6203. doi: 10.1371/journal.pone.0150171. URL <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0150171>.
- Steven L. Brunton, Marko Budišić, Eurika Kaiser, and J. Nathan Kutz. Modern Koopman Theory for Dynamical Systems. *SIAM Review*, 64(2):229–340, May 2022. ISSN 0036-1445. doi: 10.1137/21M1401243. URL <https://doi.org/10.1137/21M1401243>.
- Scott Shaobing Chen, David L. Donoho, and Michael A. Saunders. Atomic Decomposition by Basis Pursuit. *SIAM Journal on Scientific Computing*, 20(1):33–61, 1998. doi: 10.1137/S1064827596304010. URL <https://doi.org/10.1137/S1064827596304010>.
- David L. Donoho and Michael Elad. Optimally sparse representation in general (nonorthogonal) dictionaries via ℓ_1 minimization. *Proceedings of the National Academy of Sciences*, 100(5): 2197–2202, March 2003. doi: 10.1073/pnas.0437847100. URL <https://doi.org/10.1073/pnas.0437847100>.
- Mahan Fathi, Clement Gehring, Jonathan Pilault, David Kanaa, Pierre-Luc Bacon, and Ross Goroshin. Course correcting koopman representations. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=A18gWgc5mi>.
- William Gilpin. Chaos as an interpretable benchmark for forecasting and data-driven modelling. In *Advances in Neural Information Processing Systems (NeurIPS) 2021*, 2021. URL <http://arxiv.org/abs/2110.05266>.
- William Gilpin. Model scale versus domain knowledge in statistical forecasting of chaotic systems. *Physical Review Research*, 5(4):043252, December 2023. doi: 10.1103/PhysRevResearch.5.043252. URL <https://link.aps.org/doi/10.1103/PhysRevResearch.5.043252>.
- Karol Gregor and Yann LeCun. Learning Fast Approximations of Sparse Coding. In *Proceedings of the 27th International Conference on Machine Learning, ICML’10*, pages 399–406, Haifa, Israel, 2010. Omnipress. ISBN 978-1-60558-907-7. doi: 10.5555/3104322.3104374. URL <https://dl.acm.org/doi/abs/10.5555/3104322.3104374>.
- D. M. Grobman. Homeomorphism of systems of differential equations. *Doklady Akademii Nauk SSSR*, 128:880–881, 1959.
- Lars Grüne and Jürgen Pannek. *Nonlinear Model Predictive Control*. Communications and Control Engineering. Springer International Publishing, Cham, 2017. ISBN 978-3-319-46023-9 978-3-319-46024-6. doi: 10.1007/978-3-319-46024-6. URL <http://link.springer.com/10.1007/978-3-319-46024-6>.
- Philip Hartman. A lemma in the theory of structural stability of differential equations. *Proceedings of the American Mathematical Society*, 11(4):610–620, 1960. doi: 10.1090/S0002-9939-1960-0121542-7.

- R.E. Kalman. On the general theory of control systems. *1st International IFAC Congress on Automatic and Remote Control, Moscow, USSR, 1960*, 1(1):491–502, August 1960. ISSN 1474-6670. doi: 10.1016/S1474-6670(17)70094-8. URL <https://www.sciencedirect.com/science/article/pii/S1474667017700948>.
- B. O. Koopman. Hamiltonian Systems and Transformation in Hilbert Space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, May 1931. doi: 10.1073/pnas.17.5.315. URL <https://www.pnas.org/doi/10.1073/pnas.17.5.315>.
- B. O. Koopman and J. v. Neumann. Dynamical Systems of Continuous Spectra. *Proceedings of the National Academy of Sciences*, 18(3):255–263, March 1932. doi: 10.1073/pnas.18.3.255. URL <https://www.pnas.org/doi/abs/10.1073/pnas.18.3.255>.
- Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, July 2018. ISSN 00051098. doi: 10.1016/j.automatica.2018.03.046. URL <http://arxiv.org/abs/1611.03537>.
- Matthew D. Kvalheim and Shai Revzen. Existence and uniqueness of global Koopman eigenfunctions for stable fixed points and periodic orbits. *Physica D: Nonlinear Phenomena*, 425:132959, November 2021. ISSN 0167-2789. doi: 10.1016/j.physd.2021.132959. URL <https://www.sciencedirect.com/science/article/pii/S0167278921001160>.
- Yueheng Lan and Igor Mezić. Linearization in the large of nonlinear systems and Koopman operator spectrum. *Physica D: Nonlinear Phenomena*, 242(1):42–53, January 2013. ISSN 0167-2789. doi: 10.1016/j.physd.2012.08.017.
- Zexiang Liu, Necmiye Ozay, and Eduardo D. Sontag. Properties of immersions for systems with multiple limit sets with implications to learning Koopman embeddings. *Automatica*, 176:112226, June 2025. ISSN 0005-1098. doi: 10.1016/j.automatica.2025.112226. URL <https://www.sciencedirect.com/science/article/pii/S0005109825001189>.
- Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9(1):4950, November 2018. ISSN 2041-1723. doi: 10.1038/s41467-018-07210-0.
- Giorgos Mamakoukas, Maria Castano, Xiaobo Tan, and Todd Murphey. Local Koopman Operators for Data-Driven Control of Robotic Systems. In *Robotics: Science and Systems XV*. Robotics: Science and Systems Foundation, June 2019. ISBN 978-0-9923747-5-4. doi: 10.15607/RSS.2019.XV.054. URL <http://www.roboticsproceedings.org/rss15/p54.pdf>.
- D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, June 2000. ISSN 0005-1098. doi: 10.1016/S0005-1098(99)00214-9. URL <https://www.sciencedirect.com/science/article/pii/S0005109899002149>.
- Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3):309–325, 2005. doi: 10.1007/s11071-005-2824-x.
- Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, June 1996. ISSN 1476-4687. doi: 10.1038/381607a0. URL <https://doi.org/10.1038/381607a0>.

- Samuel E. Otto and Clarence W. Rowley. Linearly recurrent autoencoder networks for learning dynamics. *SIAM Journal on Applied Dynamical Systems*, 18(1):558–593, 2019. doi: 10.1137/18M1177846.
- Shaowu Pan and Karthik Duraisamy. On the lifting and reconstruction of nonlinear systems with multiple invariant sets. *Nonlinear Dynamics*, 112(12):10157–10165, June 2024. ISSN 1573-269X. doi: 10.1007/s11071-024-09581-0. URL <https://doi.org/10.1007/s11071-024-09581-0>.
- Clarence W. Rowley, Igor Mezić, Shervin Bagheri, Philipp Schlatter, and Dan S. Henningson. Spectral analysis of nonlinear flows. *Journal of Fluid Mechanics*, 641:115–127, 2009. doi: 10.1017/S0022112009992059.
- Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. ISSN 0022-1120. doi: 10.1017/S0022112010001217.
- Haojie Shi and Max Q.-H. Meng. Deep Koopman Operator With Control for Nonlinear Systems. *IEEE Robotics and Automation Letters*, 7(3):7700–7707, July 2022. ISSN 2377-3766. doi: 10.1109/LRA.2022.3184036.
- Naoya Takeishi, Yoshinobu Kawahara, and Takehisa Yairi. Learning koopman invariant subspaces for dynamic mode decomposition. In *Advances in Neural Information Processing Systems 30*, volume 30, 2017. URL <https://papers.nips.cc/paper/6713-learning-koopman-invariant-subspaces-for-dynamic-mode-decomposition>.
- Jonathan H. Tu, Clarence W. Rowley, Dirk M. Luchtenburg, Steven L. Brunton, and J. Nathan Kutz. On dynamic mode decomposition: Theory and applications. *Journal of Computational Dynamics*, 1(2):391–421, December 2014. ISSN 2158-2491. doi: 10.3934/jcd.2014.1.391. URL <https://www.aims sciences.org/en/article/doi/10.3934/jcd.2014.1.391>.
- Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A Data-Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, December 2015. ISSN 1432-1467. doi: 10.1007/s00332-015-9258-5. URL <https://doi.org/10.1007/s00332-015-9258-5>.
- Matthew O. Williams, Clarence W. Rowley, and Ioannis G. Kevrekidis. A kernel-based method for data-driven koopman spectral analysis. *Journal of Computational Dynamics*, 2(2):247–265, May 2016. ISSN 2158-2491. doi: 10.3934/jcd.2015005. URL <https://www.aims sciences.org/article/doi/10.3934/jcd.2015005>.